

JAVA MASTERY SERIES

EXCEPTION HANDLING

Chapter 1 · Complete Reference Guide

■ 12 Topics

■ 50 Theory Q&A;

■ 50 Practice Programs

■ Glossary — 30 Terms

Topics: Introduction · Hierarchy · Keywords · try-catch · finally
throw & throws · Propagation · Custom Exceptions · try-with-resources
Exception Methods · Advanced Concepts · Best Practices

■ Topics Overview

1. Introduction to Exception Handling

Exception handling is Java's built-in mechanism to detect, signal, and recover from abnormal runtime events, enabling programs to remain robust and user-friendly even when unexpected situations arise.

■ Definition of Exception

An exception is an object representing an abnormal event that disrupts the normal sequential flow of a Java program at runtime. It is an instance of a class that extends `java.lang.Throwable`, carrying details like an error message and the call stack.

- **WHAT:** Every exception is an object — it has fields (message, cause) and methods (`getMessage`, `printStackTrace`).
- **WHY:** Treating errors as objects allows them to be created, thrown, caught, and inspected just like any other object.

■ Need of Exception Handling

Real-world programs interact with files, networks, databases, and user input — all of which can fail unpredictably. Exception handling separates normal logic from error-handling code, keeping programs readable and resilient.

- **WHAT:** A structured mechanism (`try/catch/finally`) that isolates risky operations and provides alternative execution paths.
- **WHY:** Without it, every operation would need manual return-code checks, cluttering business logic and making errors easy to overlook.

■ Exception vs Error

An Exception represents a condition that a well-written application might reasonably catch and recover from (e.g., `FileNotFoundException`). An Error represents a severe JVM-level problem (e.g., `OutOfMemoryError`) that an application should not attempt to handle.

- **WHAT:** Both extend `Throwable` but form separate branches — Error and Exception.
- **WHY:** The split guides developers: handle Exceptions (recoverable); let Errors propagate or crash the app.

■ Types of Errors (Compile-time, Runtime, Logical)

Compile-time errors are syntax/semantic mistakes flagged by the compiler before the program runs. Runtime errors are exceptions thrown during execution when something goes wrong. Logical errors produce incorrect output but throw no exceptions — they are the hardest to detect.

- **WHAT:** Only runtime errors are handled by Java's exception-handling machinery.
- **WHY:** Knowing the category tells you where and how to fix: compiler for compile-time, try-catch for runtime, debugging for logical.

2. Exception Hierarchy

Java's entire exception system is a class hierarchy rooted at `java.lang.Throwable`, providing a structured taxonomy for every possible error condition in the JVM.

■ Throwable Class

Throwable is the superclass of ALL errors and exceptions. Only Throwable instances can be thrown or caught. Key methods: `getMessage()`, `printStackTrace()`, `getCause()`, `getSuppressed()`.

- WHAT: Acts as the universal type for anything that can be thrown.
- WHY: A single root lets `catch(Throwable t)` intercept everything, and provides a common API for inspecting any thrown object.

■ Error Class

Error extends Throwable and represents serious, unrecoverable JVM-level failures: `OutOfMemoryError`, `StackOverflowError`, `VirtualMachineError`. Applications must NOT try to catch these.

- WHAT: Signals JVM infrastructure failures outside application control.
- WHY: Attempting recovery from `OutOfMemoryError` is pointless and potentially dangerous — let the JVM terminate.

■ Exception Class

Exception extends Throwable and is the root of all application-level exceptional conditions. Splits into `RuntimeException` (unchecked) and all other Exceptions (checked).

- WHAT: The parent of every exception developers write, catch, and throw.
- WHY: Provides the branching point that determines whether the compiler enforces handling (checked) or not (unchecked).

■ Checked vs Unchecked Exceptions

Checked exceptions (extending `Exception` but not `RuntimeException`) must be explicitly handled or declared. Unchecked exceptions (extending `RuntimeException` or `Error`) do not require handling. The compiler enforces this at compile time.

- WHAT: Checked = compiler-enforced; Unchecked = programmer-disciplined.
- WHY: Checked exceptions model predictable, recoverable failures (`IOException`); unchecked model programming bugs (`NullPointerException`).

■ Common Exceptions

`ArithmeticException` ($\div 0$), `NullPointerException` (null dereference), `ArrayIndexOutOfBoundsException` (bad index), `ClassCastException` (invalid cast), `NumberFormatException` (bad parse), `IOException` (I/O failure), `SQLException` (DB error).

- WHAT: Each is a specific subclass with a meaningful name and message.
- WHY: Specific exception types allow fine-grained catch blocks and targeted recovery strategies.

3. Keywords in Exception Handling

Java provides five dedicated keywords that form the complete grammar of its exception-handling model: `try`, `catch`, `finally`, `throw`, and `throws`.

■ try block

Encloses code that might throw an exception. The JVM monitors this block and transfers control to the matching catch block if an exception is thrown.

- WHAT: Marks the boundary of "risky" code.
- WHY: Without `try`, the JVM has no target to monitor; exceptions would always propagate unhandled.

■ catch block

Follows a try block and handles a specific exception type. The caught exception object is available as the block's parameter for inspection and recovery.

- WHAT: The handler — defines what to do when a specific exception occurs.
- WHY: Allows different recovery strategies for different exception types.

■ finally block

Always executes after try and any catch, regardless of whether an exception occurred or was caught. The standard location for cleanup code (closing streams, releasing locks).

- WHAT: The guaranteed cleanup zone.
- WHY: Prevents resource leaks in both normal and exceptional code paths.

■ throw keyword

Used inside a method to explicitly create and throw an exception: `throw new IllegalArgumentException("reason")`. Immediately transfers control out of the current method.

- WHAT: The action of signaling an error condition.
- WHY: Allows methods to communicate failure to callers programmatically.

■ throws keyword

Appears in a method signature to declare that the method may propagate one or more checked exceptions: `void read() throws IOException`. Forces callers to acknowledge potential failure.

- WHAT: Part of a method's contract — a warning to callers.
- WHY: Makes failure modes explicit in the API, enabling callers to write appropriate handling code.

4. try-catch Mechanism

The try-catch mechanism is Java's primary tool for intercepting and handling exceptions, supporting multiple variants for different complexity levels.

■ Basic try-catch Syntax

`try { risky code } catch (ExceptionType e) { handle }`. The catch block's parameter `e` provides `getMessage()`, `printStackTrace()`, and `getCause()` for diagnosis.

- WHAT: The simplest form — one try, one catch.
- WHY: The JVM generates an exception table per method that maps try-block PC ranges to handler PCs.

■ Multiple catch Blocks

A single try block may have multiple catch blocks, each targeting a different exception type. Java checks them top-to-bottom and executes the first matching one only.

- WHAT: Multiple handlers for different failure modes.
- WHY: Different exceptions warrant different recovery strategies; one generic handler loses specificity.

■ Order of catch Blocks

More specific (child) exceptions must be placed before more general (parent) ones. Placing `IOException` before `FileNotFoundException` causes a compile-time "already caught" error.

- WHAT: Specific-to-general ordering rule.
- WHY: Java checks top-to-bottom — a parent catch would always match, making child catches unreachable.

■ Nested try-catch

A try-catch block placed inside another try or catch block allows fine-grained handling for a sub-operation within a broader operation.

- WHAT: Exception handling within exception handling.
- WHY: Some inner operations may need their own specific recovery without abandoning the outer operation.

■ try without catch / try without finally

A try block can omit catch if a finally is present (cleanup only). A try-catch without finally is valid when no cleanup is needed. At least one of catch or finally must exist.

- WHAT: Flexible try syntax — catch and finally are individually optional but not both.
- WHY: Supports cleanup-only (try-finally) and handle-only (try-catch) patterns separately.

5. finally Block

The finally block provides an unconditional execution guarantee — its code runs after try/catch regardless of how the block exits, making it the safest place for cleanup.

■ Purpose of finally

Guarantees execution of cleanup code — closing files, releasing DB connections, unlocking mutexes — even when exceptions bypass normal flow.

- WHAT: The cleanup guarantee zone.
- WHY: Without finally, every exception path would require duplicated cleanup code.

■ Execution Cases

finally runs: (1) after normal completion of try, (2) after a matching catch, (3) even if no catch matches (runs before the exception propagates), (4) even if try/catch has a return statement.

- WHAT: All four scenarios result in finally executing.
- WHY: Its unconditional nature is the whole point — it is the most reliable place for resource cleanup.

■ finally with return Statement

If both try and finally contain return statements, the finally return overrides the try return — the try's value is discarded. This is confusing behavior and considered bad practice.

- WHAT: finally's return wins, silently discarding try's return.
- WHY: The JVM stores try's return value, then executes finally; if finally returns, the stored value is abandoned.

■ finally Not Executing Cases

finally is skipped only when: System.exit() is called, the JVM crashes (SIGSEGV, OOM kill), or the thread is forcibly killed. These are outside normal Java control flow.

- WHAT: Extremely rare cases where the JVM itself cannot continue.
- WHY: If the process is gone, there is nowhere to execute finally — it is a physical impossibility.

6. throw and throws

throw and throws are complementary keywords: throw performs the action of throwing an exception; throws declares the possibility of propagation.

■ Difference: throw vs throws

throw is a statement inside a method body that throws an exception instance right now. throws is a declaration in a method signature listing checked exceptions the method may propagate.

- WHAT: throw = action (runtime); throws = declaration (compile-time contract).
- WHY: Together they form a complete exception system — signaling at the throw site and declaring at the method boundary.

■ User-Defined Throwing

throw new InsufficientFundsException("Balance too low") signals a domain-specific error. Any Throwable subclass can be thrown, enabling rich, meaningful error objects.

- WHAT: Explicitly creating and signaling an error condition in domain language.
- WHY: Generic exceptions (IllegalArgumentException) lack domain context; custom ones are self-documenting.

■ Declaring Exceptions using throws

void transferFunds() throws InsufficientFundsException, BankException {} — callers see both possible failures and must handle or re-declare them.

- WHAT: The method's failure contract visible to all callers.
- WHY: Only checked exceptions need declaration; unchecked exceptions (RuntimeException) may be listed optionally for documentation.

■ Propagation using throws

When a method declares throws, the exception travels up the call stack frame by frame until a matching catch is found or the JVM's default handler terminates the thread.

- WHAT: The mechanism by which exceptions travel from origin to handler.
- WHY: Intermediate methods can propagate without handling, keeping code clean; only the method with the right context handles it.

7. Exception Propagation

Exception propagation is the automatic bubbling of an unhandled exception up through the call stack, executing finally blocks along the way, until a handler is found or the thread terminates.

■ Call Stack Mechanism

Each method call adds a frame to the call stack. When an exception is thrown, the JVM checks the current frame's exception table. If no handler, the frame is popped and the exception moves to the caller's frame.

- WHAT: Frame-by-frame traversal searching for a handler.
- WHY: Allows high-level code to handle failures from deep low-level operations without every intermediate method needing to catch.

■ Stack Unwinding

As an exception propagates, each popped frame's finally blocks execute in order — this is stack unwinding. It guarantees cleanup at every layer even as the exception travels upward.

- WHAT: The process of removing frames while running their finally blocks.
- WHY: Ensures every method participating in the call chain gets a chance to release its resources.

■ Method Chaining

When methods are chained: `a().b().c()`, an exception in any method propagates straight out of the entire expression. The try-catch must wrap the whole chain to intercept it.

- WHAT: Chained calls are one expression — one exception exits the whole chain.
- WHY: No intermediate method call in the chain catches it; the exception must be caught at the level that can respond.

8. Custom Exceptions

Custom exceptions let developers model domain-specific failure conditions with meaningful names, additional data fields, and appropriate checked/unchecked semantics.

■ Creating User-Defined Exception

Define a class extending `Exception` or `RuntimeException`. Add constructors that pass a message to `super()`. Optionally add domain-specific fields (`errorCode`, `accountId`).

- WHAT: A new exception class tailored to a specific domain failure mode.
- WHY: Self-documenting — `InsufficientFundsException` conveys more meaning than `IllegalStateException`.

■ Extending Exception (Checked)

Extending `Exception` creates a checked custom exception — the compiler forces callers to handle or declare it. Best for recoverable, expected failures.

- WHAT: Checked custom exception — compile-time enforcement of handling.
- WHY: Use when callers are expected to meaningfully recover (e.g., retry, notify user).

■ Extending RuntimeException (Unchecked)

Extending `RuntimeException` creates an unchecked exception — no forced handling. Preferred by modern frameworks (Spring) for domain exceptions that represent unrecoverable programming errors.

- WHAT: Unchecked custom exception — no boilerplate propagation required.
- WHY: Reduces code clutter in layered architectures where checked exceptions cause cascading throws declarations.

9. try-with-resources (Java 7+)

`try-with-resources` automatically closes any `AutoCloseable` resource declared in the try parentheses when the block exits — normally or via exception — eliminating manual finally cleanup.

■ Auto Resource Management

`try (BufferedReader br = new BufferedReader(new FileReader(path))) { ... }` — `br.close()` is called automatically. No finally block needed.

- WHAT: The JVM generates a synthetic finally block that calls `close()`.
- WHY: Replaces verbose and error-prone try-finally patterns with clean, safe syntax.

■ AutoCloseable Interface

Any class implementing `AutoCloseable` (single `close()` method) qualifies for `try-with-resources`. The standard `Closeable` extends `AutoCloseable` for I/O resources.

- WHAT: The contract required for participation in `try-with-resources`.

- WHY: A standard interface allows the compiler to generate `close()` calls without knowing the specific type.

■ Multiple Resources

`try (ResourceA a = new ResourceA(); ResourceB b = new ResourceB()) {}` — both are closed automatically in reverse declaration order (B first, then A).

- WHAT: Multiple resources in one try, closed in LIFO order.
- WHY: Reverse order mirrors typical dependency: open A, open B (which depends on A) → close B before A.

10. Exception Handling Methods

The `Throwable` class provides key methods for extracting information about an exception, essential for logging, debugging, and user feedback.

■ `printStackTrace()`

Prints the exception class name, message, and full call-stack frames to `System.err`. Each frame shows class name, method name, source file, and line number.

- WHAT: The most detailed output for understanding what went wrong and where.
- WHY: Reveals the complete path from the throw site through all propagating frames.

■ `getMessage()`

Returns the detail message `String` passed to the exception constructor. Returns `null` if no message was provided.

- WHAT: The human-readable reason for the exception.
- WHY: Used to display meaningful error messages to users or in logs.

■ `toString()`

Returns a compact description: `getClass().getName() + ": " + getMessage()`. Example: `java.lang.ArithmeticException: / by zero`.

- WHAT: A one-line summary identifying the exception type and message.
- WHY: Convenient for quick logging or displaying in UIs.

11. Advanced Concepts

Java 7 introduced several advanced exception-handling features that handle complex, real-world scenarios more elegantly.

■ Multi-catch (Java 7)

`catch (IOException | SQLException e) {}` catches two unrelated exception types with identical handling in a single block. The variable `e` is implicitly `final`.

- WHAT: One catch block, multiple exception types.
- WHY: Eliminates duplicated catch blocks with identical handling logic; enforces that the types are not in a parent-child relationship.

■ Chained Exceptions

`throw new ServiceException("Msg", originalException)` wraps a low-level exception as the cause of a high-level one, preserving the full diagnostic chain. `getCause()` retrieves the wrapped exception.

- WHAT: Wrapping low-level exceptions inside domain-level ones without losing context.
- WHY: Abstraction layers can translate exception types while preserving the original root-cause for debugging.

■ Suppressed Exceptions

In try-with-resources, if `close()` throws while a primary exception exists, the `close()` exception is attached to the primary via `addSuppressed()` rather than replacing it.

- WHAT: Secondary exceptions preserved alongside the primary, not discarded.
- WHY: Prevents silent loss of diagnostic information when both the try body and `close()` fail.

■ Rethrowing Exceptions

Catch an exception, perform partial handling (logging), then rethrow: `catch(IOException e){ log(e); throw e; }` — since Java 7, the compiler tracks the specific rethrown type.

- WHAT: Partial handling at one layer, final handling at another.
- WHY: Enables logging middleware to observe exceptions without owning the recovery logic.

12. Best Practices

Following these best practices leads to robust, maintainable exception handling that is easy to debug in production and clean to read in code review.

■ Avoid Empty catch Blocks

`catch(Exception e) {}` silently swallows exceptions — bugs become invisible. Always log or rethrow. The minimum: `logger.error("Unexpected", e)`.

- WHAT: The most dangerous anti-pattern in exception handling.
- WHY: Silent failure makes issues impossible to diagnose; problems compound silently over time.

■ Use Specific Exceptions

Catch `IOException` specifically rather than `Exception` broadly. Each catch should handle only the exception types it can meaningfully recover from.

- WHAT: Fine-grained catch targeting.
- WHY: Over-broad catches may mask unrelated errors and make refactoring dangerous.

■ Resource Handling

Always use try-with-resources for `Closeable`/`AutoCloseable` resources. Never rely on GC or finalization to close streams.

- WHAT: Use language-level resource management.
- WHY: Manual finally blocks are error-prone; try-with-resources handles even the case where `close()` itself throws.

■ Do Not Overuse Exceptions

Exceptions are for exceptional conditions — not normal control flow. Do not catch `NumberFormatException` to detect non-numeric input; use `Integer.parseInt()` inside a conditional after `isDigit()` checks.

- WHAT: Reserve exceptions for unexpected, rare, unrecoverable-inline conditions.
- WHY: Exception creation (stack trace capture) is 100-10,000× slower than a conditional check.

■ Theoretical Questions & Answers

■ EASY — Conceptual Foundation (1–4 sentences per answer)

Q1. What is an exception in Java?

An exception is an unwanted or unexpected runtime event that disrupts the normal sequential flow of a Java program. It is represented as an object — an instance of a `Throwable` subclass — carrying the error message and the call stack at the point of occurrence.

Q2. What is an Error in Java?

An Error represents a serious, generally unrecoverable, JVM-level problem such as `OutOfMemoryError` or `StackOverflowError`. Applications should not attempt to catch or recover from Errors, as they indicate the JVM itself is in an unstable state.

Q3. What is the difference between an Error and an Exception?

Both extend `Throwable`, but Errors represent severe JVM-level failures (unrecoverable) while Exceptions represent application-level problems that can often be caught and handled. Errors should not be caught; Exceptions frequently are.

Q4. What is a try block?

A try block wraps code that might throw an exception. If an exception occurs inside it, normal execution stops and control immediately transfers to the matching catch block or the finally block.

Q5. What is a catch block?

A catch block follows a try block and handles a specific type of exception. It receives the thrown exception object as a parameter, allowing the handler to inspect the error message, log it, or perform recovery.

Q6. What is a finally block?

A finally block always executes after the try and any catch blocks, regardless of whether an exception was thrown or caught. It is the standard location for cleanup code like closing files or database connections.

Q7. What is the throw keyword?

`throw` is a statement that explicitly throws an exception object from within a method: `throw new IllegalArgumentException('Bad input')`. Execution immediately leaves the current method after `throw`.

Q8. What is the throws keyword?

`throws` appears in a method signature to declare that the method may propagate one or more checked exceptions: `void readFile() throws IOException`. Callers must either handle these exceptions or re-declare them.

Q9. What is a checked exception?

A checked exception extends `Exception` but not `RuntimeException`. The compiler mandates that it be either caught with try-catch or declared in the method's throws clause. Examples: `IOException`, `SQLException`.

Q10. What is an unchecked exception?

An unchecked exception extends `RuntimeException` (or `Error`). The compiler does not require explicit handling or declaration. Examples: `NullPointerException`, `ArrayIndexOutOfBoundsException`, `ArithmeticException`.

Q11. Give an example of a RuntimeException.

`NullPointerException` is the classic example — thrown when code tries to call a method or access a field on a null object reference, e.g., `String s = null; s.length();` throws `NullPointerException`.

Q12. Give an example of a checked exception.

`IOException` is a checked exception thrown by file and network I/O operations. Every method that reads from or writes to a file must either catch `IOException` or declare throws `IOException`.

Q13. What is a stack trace?

A stack trace is a textual report of the active method-call chain at the moment an exception occurred, listing each method frame (class name, method name, source file, line number) from the throw site upward.

Q14. What happens if an exception is not handled?

If an exception propagates through the entire call stack without being caught, the JVM invokes the thread's uncaught exception handler, which by default prints the exception type, message, and stack trace to `System.err` and terminates the thread.

Q15. Can we use try without catch?

Yes — a try block can omit catch if it has a finally block: `try { ... } finally { ... }`. The finally guarantees cleanup even without a handler. At least one of catch or finally must accompany every try.

Q16. Can we use try without finally?

Yes — `try { ... } catch (Exception e) { ... }` with no finally is perfectly valid. The finally block is optional as long as at least one catch block is present to handle the exception.

Q17. What is a multiple catch block?

Multiple catch blocks after a single try allow each exception type to be handled differently. Java matches the thrown exception to catch blocks from top to bottom and executes the first matching one only.

Q18. What is nested try-catch?

A nested try-catch is a try-catch block placed inside another try or catch block, enabling more granular exception handling for a specific sub-operation within a larger try-protected section.

Q19. What is exception propagation?

Exception propagation is the automatic process by which an unhandled exception travels up the call stack — from the method where it was thrown to its caller, and so on — until a catch block handles it or the thread terminates.

Q20. What is the Throwable class?

Throwable is the root of Java's entire exception hierarchy. Only Throwable instances can be thrown or caught. It provides `getMessage()`, `printStackTrace()`, `getCause()`, and `getSuppressed()` — the core API for all exception inspection.

■ ■ MEDIUM — In-Depth Understanding (4–8 sentences per answer)**Q1. What is the difference between throw and throws?**

throw is a runtime statement inside a method body that actually launches an exception object: `throw new IOException('File missing')`. throws is a compile-time keyword in a method signature that announces which checked exceptions the method may propagate: `void load() throws IOException`. throw takes an object reference; throws takes class names. You can throw only one exception at a time, but declare multiple in throws. throw is mandatory to actually signal the error; throws is the contract warning to callers. Unchecked exceptions can be thrown without being listed in throws. Missing throws for a checked exception is a compile error if the exception is not caught locally.

Q2. Explain checked vs unchecked exceptions in detail.

Checked exceptions extend `Exception` directly (excluding `RuntimeException`) and are verified by the compiler — every call site must either catch them or re-declare them in its own throws clause. This is sometimes called 'exception safety by contract.' Examples include `IOException` (file I/O), `SQLException` (database), `ClassNotFoundException` (class loading). Unchecked exceptions extend `RuntimeException` and represent programming mistakes that developers are expected to prevent rather than handle: `NullPointerException` (null dereference), `ArrayIndexOutOfBoundsException` (index logic), `ClassCastException` (bad cast), `NumberFormatException` (parse error). The Java compiler does not verify handling of unchecked exceptions. Checked exceptions improve robustness in external integrations but add boilerplate. Many modern frameworks (Spring, Hibernate) prefer unchecked custom exceptions to avoid propagation noise. The debate over checked vs unchecked exceptions is one of the longest-running in the Java community.

Q3. How does the JVM handle exceptions internally?

When bytecode is compiled, the Java compiler generates an exception table for each method, embedded in the .class file. Each entry maps a PC (program counter) range (`start_pc` to `end_pc`) to a `handler_pc` and a matched exception type. When the JVM executes a throw statement (or an implicit throw from a null dereference, bad cast, etc.), it searches the current method's exception table for an entry where the current PC falls in range and the thrown type matches (via instanceof semantics). If found, the operand stack is cleared, the exception object is pushed, and execution jumps to `handler_pc` (the catch block). If not found, any finally blocks in the current method execute, the current stack frame is popped (stack unwinding), and the process repeats in the caller's frame. This continues until a handler is found or the thread's uncaught exception handler is invoked. The entire try-block setup adds zero overhead at runtime — cost is only incurred when an exception is actually thrown (primarily the stack trace capture in `fillInStackTrace()`).

Q4. Why does the order of catch blocks matter?

Java evaluates catch blocks sequentially from top to bottom, executing the first block whose declared type matches the thrown exception via instanceof. Because exceptions form an inheritance hierarchy, a parent-type catch (e.g., `Exception`) would match any exception, including subtypes. If `Exception` is placed before `IOException`, the `IOException` catch block would become unreachable — the compiler detects this and reports 'Exception already caught' as a compile error. The rule: always order catch blocks from most specific (child) to most general (parent). For example: `catch(FileNotFoundException)`

before `catch(IOException)` before `catch(Exception)`. In multi-catch (Java 7), the listed types must not be in a parent-child relationship.

Q5. What is multi-catch in Java 7 and when do you use it?

Multi-catch allows a single catch block to handle multiple unrelated exception types using the pipe operator: `catch (IOException | SQLException e)`. Both types are handled identically within the block. The variable `e` is implicitly final — you cannot reassign it inside the block, preventing accidental type inconsistencies. Multi-catch only works for exception types that are not in a supertype-subtype relationship; catching two types where one extends the other is a compile error. The benefit is eliminating duplicate catch blocks with identical handling logic, which is both concise and reduces copy-paste maintenance errors. Multi-catch is particularly useful in DAO or service layers where multiple persistence-related exceptions need the same logging and re-wrapping treatment.

Q6. Explain the finally block with a return statement.

If a try block contains a return statement, the JVM temporarily stores the return value, then executes the finally block before actually returning. If the finally block also contains a return statement, it completely overrides the try's return — the stored value is discarded, and finally's value is returned instead. For example: `try { return 1; } finally { return 2; }` — the method returns 2. This behavior is widely considered a gotcha and is classified as bad practice. It can silently discard computed return values and cause subtle bugs that are extremely hard to track down. The recommendation is to never use `return`, `break`, `continue`, or `throw` inside a finally block, as these suppress exceptions from the try/catch body.

Q7. What is try-with-resources and how does it work internally?

Introduced in Java 7, try-with-resources automates resource cleanup for any class implementing `AutoCloseable`. The syntax is: `try (Resource r = new Resource()) { ... }`. The compiler generates a synthetic finally block that calls `r.close()` when the try block exits — whether normally, via return, or via exception. Multiple resources can be declared, separated by semicolons; they are closed in reverse declaration order. The key improvement over manual try-finally is the handling of suppressed exceptions: if the try body throws exception A and `close()` throws exception B, B is attached to A as a suppressed exception (via `addSuppressed()`) rather than replacing A. This preserves full diagnostic information. `getSuppressed()` retrieves the suppressed exceptions from A.

Q8. What is the difference between final, finally, and finalize()?

`final` is a keyword with three uses: applied to a variable, it makes it a constant (cannot be reassigned); applied to a method, it prevents overriding in subclasses; applied to a class, it prevents subclassing entirely. `finally` is a block in a try-catch statement that always executes, used for resource cleanup. `finalize()` is an instance method inherited from `Object` that the garbage collector was specified to call before reclaiming an unreachable object. `finalize()` is deprecated since Java 9 because it is unpredictable (no guarantee of when or whether it runs), can delay GC, and can cause resource leaks. All three share the 'final' prefix but serve completely unrelated purposes: immutability/inheritance enforcement, exception flow control, and (defunct) GC lifecycle notification respectively.

Q9. What is a custom exception and why do we create one?

A custom exception is a developer-defined class that extends `Exception` or `RuntimeException` to model a domain-specific failure condition. The minimum implementation needs only constructors delegating to

`super(message)` and optionally `super(message, cause)`. More sophisticated custom exceptions add domain-specific fields (`errorCode`, `transactionId`, `accountNumber`) with getters, enabling callers to extract structured error context. Custom exceptions make code self-documenting: `InsufficientFundsException` communicates far more than `IllegalStateException`. They also enable callers to catch specific domain errors and take targeted actions (retry, compensate, alert). When building APIs or libraries, custom exceptions are considered essential for clean abstraction — callers should not be exposed to implementation-specific exceptions like `SQLException`.

Q10. Can a constructor throw an exception? Explain.

Yes, Java constructors can declare `throws` for checked exceptions and throw any exception including unchecked ones. If a constructor cannot complete initialization due to invalid arguments, resource failures, or constraint violations, throwing an exception is the cleanest way to signal this. For example, a constructor opening a network connection might throw `IOException` if the connection fails. When a constructor throws, the object is never fully initialized and is not assigned to the variable — the partially constructed object becomes eligible for garbage collection. Callers must handle the exception as with any method. Checked exceptions thrown from constructors must be declared in the constructor's `throws` clause. This pattern is preferable to returning a partially initialized 'invalid' object that callers must check separately.

Q11. What happens if an exception is thrown inside a finally block?

If code inside a finally block throws an exception, it REPLACES the original exception from the try block — the original exception is completely lost (unlike `try-with-resources` which uses suppressed exceptions). For example: `try { throw new IOException(); } finally { throw new RuntimeException(); }` — only `RuntimeException` propagates; `IOException` is silently discarded. This is one of the most dangerous behaviors in Java exception handling because it can make the root cause of a problem permanently invisible. This is exactly why `try-with-resources` is superior to manual finally for resource cleanup — it uses `addSuppressed()` to preserve both exceptions. Best practice: always guard cleanup code inside finally with its own try-catch to prevent the primary exception from being swallowed.

Q12. How do you create a user-defined checked exception?

Step 1: Define a class extending `Exception` (not `RuntimeException`). Step 2: Add at minimum two constructors — a message constructor (`public MyException(String msg) { super(msg); }`) and a cause constructor (`public MyException(String msg, Throwable cause) { super(msg, cause); }`). Step 3: Optionally add domain-specific fields and getters: `private final int errorCode; public int getErrorCode()`. Step 4: Declare it in `throws` clauses of methods that throw it: `void process() throws MyException`. Step 5: Throw with `new`: `throw new MyException('Reason', 1001)`. Following naming conventions (suffix `Exception`), providing both constructors (for chaining), and adding meaningful fields constitute a production-quality custom exception.

Q13. What is rethrowing an exception and when do you use it?

Rethrowing means catching an exception and then throwing it again (the same object or a wrapped one) so a higher layer can handle it. The simplest form: `catch(IOException e) { logger.error('IO failed', e); throw e; }` — the exception propagates after logging. Since Java 7, the compiler tracks the specific types throwable in a block: if the try can only throw `IOException` and you catch `Exception` and rethrow `e`, the compiler knows the actual type is `IOException` and allows declaring `throws IOException` in the signature.

rather than throws Exception. Alternative rethrowing wraps in a new exception: throw new ServiceException('Service failed', e) — this translates the exception type while preserving the cause chain. Use rethrowing when a method can partially respond (log, update metrics) but cannot fully recover from the failure.

Q14. What are chained exceptions and why are they important?

Chained exceptions (introduced in Java 1.4) allow wrapping a lower-level exception inside a higher-level one, forming a diagnostic chain. Constructor: new ServiceException('DB failure', sqlException). The original exception is stored as the 'cause' and retrieved with getCause(). Chaining solves the abstraction problem: a service layer should throw ServiceException, not expose SQLException to callers (which would leak implementation details). But without chaining, the root cause would be lost, making debugging impossible. printStackTrace() prints the full chain: the outer exception first, then 'Caused by:' entries for each cause in the chain. getCause() can be called repeatedly to traverse the chain programmatically. Deep chains (4+ levels) are possible in layered architectures and are invaluable when debugging production issues through log files.

Q15. What are the rules for exceptions in method overriding?

Java enforces strict rules on exceptions in overriding methods, derived from the Liskov Substitution Principle. An overriding method (in a subclass): (1) CANNOT declare new checked exceptions not in the parent's throws clause. (2) CANNOT declare broader checked exceptions (e.g., if parent throws IOException, override cannot throw Exception). (3) CAN declare fewer checked exceptions, narrower ones, or none. (4) CAN declare any unchecked exceptions freely. If the parent method throws IOException, the override can throw FileNotFoundException (narrower), declare no exceptions, or throw any RuntimeException. The rationale: code holding a supertype reference must not be surprised by new checked exceptions from a subtype — that would break polymorphic use. Violation causes a compile-time error.

Q16. What are suppressed exceptions?

Suppressed exceptions are secondary exceptions that are 'attached' to a primary exception using addSuppressed(Throwable) rather than replacing it. This mechanism was introduced in Java 7 specifically for try-with-resources. When a try-with-resources block has both a body exception and a close() exception, the close() exception is suppressed and attached to the body exception. Callers retrieve them with getSuppressed(), which returns a Throwable[]. In stack traces, suppressed exceptions appear with the label 'Suppressed:' indented under the primary exception. This solves the data-loss problem of the old try-finally pattern where a finally exception would silently replace the original exception. Developers can also manually call addSuppressed() in custom resource management code.

Q17. What are the best practices for exception handling?

1. Never empty-catch: always log or rethrow.
2. Catch specifically: use the most specific exception type applicable.
3. Use try-with-resources for all Closeable resources — prevents leaks.
4. Log with context: include method name, input data, and the full exception (not just the message).
5. Do not use exceptions for control flow — they are expensive (stack trace capture).
6. Provide meaningful messages when throwing: explain what was invalid and why.
7. Prefer unchecked custom exceptions in modern layered architectures to reduce propagation boilerplate.
8. Chain exceptions with cause when re-throwing at abstraction boundaries.
9. Never catch Throwable or Error unless you have a very

specific, well-understood reason. 10. Document thrown exceptions with `@throws` Javadoc. 11. Handle exceptions at the layer that can meaningfully respond to them.

Q18. Why is `RuntimeException` unchecked?

`RuntimeException` and its subclasses represent programming errors — bugs in the code itself rather than external failure conditions. The Java designers reasoned: if `NullPointerException` were checked, every method call (which could potentially produce one) would require a try-catch or throws declaration, making all Java code unreadable. Programmers are expected to prevent these conditions through defensive coding (null checks, bounds validation) rather than by catching them. Checked exceptions model expected, recoverable failures in external interactions (network, files, databases). Unchecked exceptions model contract violations and bugs that should be fixed in code, not recovered from at runtime. This distinction keeps Java code readable for the common case (no NPE handling) while enforcing contract acknowledgment for truly exceptional external interactions.

Q19. Can the main method throw an exception?

Yes — `main` can declare throws `Exception`: `public static void main(String[] args) throws Exception`. If `main` throws, the JVM's default uncaught exception handler catches it at the top level, prints the exception class name, message, and full stack trace to `System.err`, and then terminates the JVM. This is acceptable for small programs, test utilities, and command-line tools where a crash with a stack trace is acceptable output. For production applications, `main` should wrap the application startup in a try-catch, handle startup failures gracefully (log, display user-friendly message), and exit with a meaningful error code via `System.exit(1)` rather than exposing a raw stack trace.

Q20. Explain exception propagation with a concrete example.

Consider: `main()` calls `orderService.placeOrder()`, which calls `paymentService.charge()`, which calls `bankApi.debit()`. If `bankApi.debit()` throws `IOException` and none of the intermediate methods have a matching catch: (1) `IOException` propagates to `paymentService.charge()`. (2) `charge()`'s exception table has no match; its finally block (if any) runs; the frame is popped. (3) `IOException` propagates to `orderService.placeOrder()` — same treatment. (4) `IOException` propagates to `main()`. (5) If `main()` has `catch(IOException e) { ... }`, it handles it there. Otherwise, the JVM's default handler prints the stack trace showing all four frames. The stack trace preserved in the exception object reflects the original throw site in `bankApi.debit()`, not the propagation path.

Q21. What is the difference between compile-time and runtime exceptions?

Compile-time exceptions are another name for checked exceptions — they are verified at compile time by the Java compiler, which refuses to compile code that neither catches nor declares them. Runtime exceptions (`RuntimeException` subclasses) are only detectable at runtime when the incorrect code path is actually executed. Compile-time checking provides early detection for foreseeable external failures; runtime exceptions surface programming bugs that slipped through code review. The terms can also refer more broadly to types of errors: compile-time errors (syntax errors, type mismatches) vs. runtime exceptions (anything thrown during JVM execution).

■ HARD — Expert-Level Analysis (8+ sentences per answer)

Q1. Explain the JVM exception-handling mechanism in depth.

When Java source code is compiled, the compiler generates an exception table for each method and embeds it in the .class file as part of the Code attribute. Each entry in this table has four components: start_pc (start of the guarded bytecode range), end_pc (exclusive end), handler_pc (where the catch block begins), and catch_type (the exception class to match, or 0 meaning 'all' for finally blocks). When the JVM executes a throw instruction (athrow bytecode) — triggered by an explicit throw statement or an implicit exception (null dereference causes aconst_null + athrow; array bounds triggers arraylength + check) — it performs a linear search through the current method's exception table: 1. Is the current PC between start_pc and end_pc? 2. Is the thrown exception an instance of catch_type (using instanceof semantics, including subclass check)? 3. If both, execution jumps to handler_pc — the catch block. The operand stack is cleared and the exception object is pushed. If no entry matches, the JVM executes any 'catch_type = 0' entries (finally blocks), pops the current stack frame (stack unwinding), and re-raises the exception in the calling method's frame. This repeats until a match is found or the bottom of the call stack is reached, at which point Thread.getUncaughtExceptionHandler().uncaughtException() is invoked. Key performance fact: The try block itself has ZERO runtime overhead — no branch instruction, counter, or register is used to mark "inside try." The exception table is consulted only when an exception is actually thrown. This is the "zero-cost exception model." The dominant cost is fillInStackTrace(), which walks the JVM's internal frame list and allocates StackTraceElement[] objects — roughly 100–300µs for a 50-frame stack on modern hardware. HotSpot optimization: For frequently-thrown exceptions in hot loops (NullPointerException, ArrayIndexOutOfBoundsException), HotSpot may pre-allocate a singleton instance without a stack trace — called "fast throw" — dramatically reducing the cost. This is invisible to the programmer but means production logs may show NPEs without stack traces in hot paths.

Q2. Explain the stack unwinding process in Java.

Stack unwinding is the orderly process by which the JVM removes stack frames as an unhandled exception propagates from its throw site toward a handler. Detailed flow: Suppose methodA() calls methodB(), which calls methodC(). methodC() throws IOException. The JVM checks methodC()'s exception table — no handler. It executes methodC()'s finally blocks (catch_type=0 entries) in order. methodC()'s frame is popped from the call stack. The IOException is now "active" in methodB()'s frame. The JVM checks methodB()'s table — no match. methodB()'s finally blocks execute. methodB()'s frame is popped. The exception reaches methodA(). If methodA() has catch(IOException e), execution transfers to that handler — unwinding stops. If not, methodA()'s finally runs and its frame is popped. This continues up through every frame in the call stack. The exception object itself is NOT modified during unwinding — its stack trace (captured once at throw time by fillInStackTrace()) always reflects the original throw site in methodC(), not the unwinding path. Synchronized block interaction: If any unwound frame was executing inside a synchronized block when the exception occurred, the JVM automatically releases the monitor (the intrinsic lock) for that object. This is guaranteed by the JVM spec and prevents deadlocks when exceptions escape synchronized sections. Local variable GC: As each frame is popped, all local variables in that frame become unreachable. Objects referenced only by those locals become eligible for garbage collection. Resources NOT protected by finally or try-with-resources are thus leaked until the next GC cycle — often for seconds in a production server. Native frames: If a native

(JNI) method is in the call stack when unwinding, the JVM cannot execute Java finally blocks for that native frame. JNI code must implement its own cleanup using RAII patterns in C/C++.

Q3. What is the performance impact of exceptions in Java?

Java exceptions carry significant performance costs that make them unsuitable for normal control flow, particularly in high-throughput systems. Primary cost — Stack trace capture: When any Throwable is instantiated (including in the constructor), it calls `fillInStackTrace()`, a synchronized native JVM method that walks the entire call stack, captures each frame (class name, method name, file name, line number) into a `StackTraceElement[]`, and stores it. For a 100-frame stack (common in web frameworks), this takes 200–500 microseconds and allocates 100+ objects plus their String data — potentially 50KB of heap per exception. Secondary costs: - Object allocation: The exception object itself plus its fields (message String, `StackTraceElement[]`, suppressed list). - Exception table lookup: Linear search in the bytecode table (negligible for small tables). - JIT deoptimization: Code compiled with optimistic assumptions (no exception paths) may deoptimize when exceptions occur, requiring recompilation with exception handling included. GC pressure: In a service throwing 100 exceptions/second, each with a 50KB trace, that's 5MB/second of short-lived garbage flooding the young generation — causing frequent minor GCs and potential GC pauses. Benchmarks (JMH): Throwing an exception is 100× to 10,000× slower than a conditional check, depending on stack depth and JIT state. HotSpot fast-throw: For common exceptions (NPE, AIOOB, ClassCast, ArithmeticException) in hot compiled methods, HotSpot may reuse a pre-allocated exception singleton without a stack trace. This trades diagnostics for performance. You'll see NPE without "at ClassName.method" in logs — a known production debugging pitfall. Mitigation: 1. Override `fillInStackTrace()` in custom signal-exceptions: `@Override public synchronized Throwable fillInStackTrace() { return this; }` — eliminates trace capture, reducing creation to microseconds. 2. Pre-allocate and reuse static exception instances for purely signaling purposes. 3. Use Optional, result types, or return codes for expected, frequent conditions. 4. Profile with async-profiler or JFR to identify exception-heavy paths.

Q4. How would you design a custom exception hierarchy for a banking system?

A well-designed exception hierarchy mirrors the domain model and enables callers to catch at the right level of abstraction. Design goals: - Make the root unchecked (extends `RuntimeException`) to avoid propagation boilerplate in service layers. - Use checked exceptions only at the outermost API boundary if contractually required. - Keep hierarchy shallow (max 3 levels) to avoid over-engineering. - Add domain-specific fields for structured error context. - Provide both message-only and message+cause constructors for chaining. Proposed hierarchy: Level 0 — Root: `BankingException` extends `RuntimeException` Fields: String `errorCode`, String `correlationId` Purpose: Base for all banking-domain exceptions; catch-all for unknown banking errors. Level 1 — Category: `AccountException` extends `BankingException` — all account lifecycle issues `TransactionException` extends `BankingException` — payment/transfer failures `AuthenticationException` extends `BankingException` — auth and session issues `ComplianceException` extends `BankingException` — regulatory/AML failures Level 2 — Specific: `InsufficientFundsException` extends `AccountException` Fields: double `available`, double `requested` Method: double `getShortfall()` { return `requested - available`; } `AccountFrozenException` extends `AccountException` Fields: String `frozenReason`, `LocalDateTime` `frozenAt` `AccountNotFoundException` extends `AccountException` (no extra fields — account ID already in the message) `DuplicateTransactionException` extends `TransactionException` Fields: String `existingTransactionId` `TransactionLimitExceededException` extends `TransactionException` Fields: double `limit`, double `requested` `InvalidCredentialsException` extends `AuthenticationException` `AccountLockedException`

extends `AuthenticationException` Fields: `int remainingAttempts`, `LocalDateTime unlockTime` Usage pattern in service layer: `catch(InsufficientFundsException e)` — trigger balance top-up notification `catch(AccountException e)` — generic account error response `catch(BankingException e)` — fallback for all banking errors `catch(Exception e)` — unexpected system error, log and alert REST error handling: `BankingException` implements `toErrorResponse()` returning a record `ErrorResponse(errorCode, message, timestamp)` for consistent JSON serialization at the API boundary.

Q5. Explain exception handling in multithreaded Java programs.

Exception handling in multithreaded Java requires special attention because exceptions are thread-local — they propagate only within the thread where they are thrown and do not cross thread boundaries automatically. Thread class and `Runnable`: When a thread created via `new Thread(runnable).start()` throws an unchecked exception in its `run()` method, the exception terminates that thread silently unless an `UncaughtExceptionHandler` is registered. The exception does NOT propagate to the thread that called `start()`. Register a handler: `thread.setUncaughtExceptionHandler((t, e) -> logger.error("Thread " + t.getName() + " died", e))`. Set a global default: `Thread.setDefaultUncaughtExceptionHandler(...)`. `ExecutorService` and `Runnable`: When a `Runnable` submitted to an `ExecutorService` throws an unchecked exception, the exception is silently caught by the executor framework, wrapped in a `Future`, and the thread is returned to the pool (or replaced). Without checking the `Future`, the exception is invisible. `executorService.submit(runnable).get()` — calling `get()` throws `ExecutionException` wrapping the original. Always call `get()` (or examine `isDone()/isCancelled()`) to detect failures. `ExecutorService` and `Callable`: `Callable.call()` can throw checked exceptions. The exception is wrapped in `ExecutionException` by the framework. `future.get()` throws `ExecutionException`; unwrap with `getCause()` to get the original. This is the correct pattern for tasks that can fail meaningfully. `ScheduledExecutorService` trap: A `ScheduledRunnable` that throws terminates its schedule silently — future executions are cancelled without any indication. Always wrap scheduled tasks in try-catch: `scheduledExecutor.scheduleAtFixedRate(() -> { try { task(); } catch(Exception e) { logger.error("Scheduled task failed", e); } }, 0, 1, TimeUnit.MINUTES)`. `CompletableFuture`: Exception handling is functional: `future.exceptionally(e -> fallback)`, `future.handle((result, e) -> ...)`, `future.whenComplete((result, e) -> ...)`. These compose cleanly in async pipelines. Unhandled `CompletableFuture` exceptions are swallowed unless `exceptionally/handle/whenComplete` is used. Structured concurrency (Java 21 preview): `StructuredTaskScope` enables exception aggregation — if any subtask fails, the scope's `shutdown()` cancels all others and `throwIfFailed()` rethrows the first failure, bringing structured error handling to concurrent code.

Q6. Explain the fail-fast vs fail-safe concept.

Fail-fast and fail-safe describe two philosophies for how systems and iterators behave when they detect an inconsistency or concurrent modification. Fail-fast philosophy: The system immediately throws an exception the moment it detects a violation, rather than continuing in a potentially corrupted or inconsistent state. The goal is early, loud failure — making problems visible immediately. In Java collections: `ArrayList`, `HashMap`, `HashSet`, and most standard collection iterators are fail-fast. They maintain an internal `modCount` field incremented on every structural modification (add, remove, clear). Iterator captures `expectedModCount` at creation. On every `next()` call, it checks `modCount == expectedModCount`. If modified, `ConcurrentModificationException` is thrown immediately. This detects programming errors (modifying a collection during iteration) rather than concurrent access — it's not a thread-safety guarantee. Fail-fast in distributed systems: Service A calls Service B. If B is down, fail-fast means A immediately returns an error to the caller rather than waiting for a timeout. Circuit breaker

pattern implements fail-fast at the system level. Fail-safe philosophy: The system continues operating even in the presence of concurrent modifications, typically by operating on a copy or snapshot. It sacrifices strict consistency for availability. In Java collections: `CopyOnWriteArrayList` and `ConcurrentHashMap` iterators are fail-safe. `CopyOnWriteArrayList`: every mutating operation creates a new copy of the underlying array. Iterators operate on the snapshot taken at iterator creation time — never see concurrent modifications, never throw `ConcurrentModificationException`. Optimal for read-heavy, write-rare scenarios. `ConcurrentHashMap`: iterators reflect the state of the map at some point during or after the iterator's creation — no CME but may not reflect very recent updates. Practical implications: Fail-fast: use when catching bugs early is critical and the collection is accessed from a single thread or under explicit external synchronization. Fail-safe: use when availability and uninterrupted iteration matter more than perfect consistency, e.g., in event-listener notification systems where listeners are iterated while new ones might be registered.

Q7. When should you NOT use exceptions?

Exceptions are powerful but are frequently misused in ways that degrade performance, readability, and system design. Here are the scenarios where exceptions should NOT be used:

1. Normal control flow: `try { return Integer.parseInt(input); } catch (NumberFormatException e) { return -1; }` Replace with: `input.matches('-?\d+') ? Integer.parseInt(input) : -1`. Exceptions are 100-10,000× slower than conditional checks.
2. Expected, frequent conditions: `if (!file.exists())` — not `try { open(file); } catch (FileNotFoundException e)`. File not found can be an expected scenario, not an exception.
3. Business rule validation: Throwing exceptions per invalid form field creates unnecessary objects and stack traces for what is a normal user interaction. Use a validation framework (Bean Validation) that returns structured error messages.
4. Null checking as normal logic: `catch (NullPointerException e)` as a substitute for null checks. NPE should NEVER be caught as a validation mechanism — it masks bugs and makes code non-obvious.
5. Performance-critical inner loops: Any exception in a hot loop (processing millions of records) with deep stacks can be catastrophic. Use return codes or `Optional` inside tight loops.
6. Testing expected behavior (before `assertThrows`): `try { doSomething(); fail(); } catch (SomeException e) {}` — use JUnit 5's `assertThrows()` instead.
7. Retry logic as the primary mechanism: Using exceptions to control retry loops: `while(true) { try { return attempt(); } catch (Exception e) { continue; } }` — use explicit retry frameworks (Resilience4j) with proper backoff and circuit breaking.
8. As a way to return multiple values: Throwing exceptions to communicate secondary results or status codes. Use return types, `Optional`, or dedicated result types (sealed classes in Java 17+).

Guiding principle: exceptions should signal EXCEPTIONAL conditions — unexpected, rare events from which the program cannot proceed inline without special handling. If a condition is frequent, expected, or part of the happy path, use explicit conditional logic instead.

Q8. Compare Exception vs Optional in Java 8.

`Optional` and exceptions both address the problem of absence or failure, but serve fundamentally different purposes and should be chosen based on the semantics of the situation.

Optional: Represents the possible ABSENCE of a value — a container that is either empty or holds exactly one value. It is a type-safe alternative to returning null. Best use: as a return type when a method may legitimately produce no result. Example: `Optional findById(long id)` — clearly signals that the user may not exist.

Functional chaining: `findById(id).map(User::getEmail).filter(email -> email.contains('@')).orElse("unknown@default.com")` — four potential null checks replaced by one clean expression.

Exceptions: Signal that something WENT WRONG — an unexpected failure, not mere absence. Carry rich diagnostic information (message, stack trace, cause chain) that `Optional` cannot.

Appropriate when absence means failure. Example: `loadConfig()` throws `ConfigNotFoundException` — absence of config is an error, not an expected possibility. Key distinctions: - Optional is zero overhead for creation; exceptions have significant creation cost (stack trace). - Optional is functional (composable with `map/flatMap`); exceptions are imperative (try-catch). - Optional should NOT be used for method parameters, fields, or within collections. - Optional can't carry an error reason; exceptions provide full context. - `Optional.orElseThrow()` bridges both — convert Optional to exception when absence is an error at the calling layer. Anti-patterns: `Optional.get()` without `isPresent()` check — just as dangerous as null dereference. Optional as a method parameter: `void process(Optional name)` — use overloading instead. Optional fields in entities — breaks serialization frameworks and JPA. Modern pattern: repositories return Optional (absence = valid empty result); service layers call `.orElseThrow()` -> `new EntityNotFoundException(id)` when the business rule requires the entity to exist — combining both semantics cleanly.

Q9. Explain the memory impact of exceptions.

Java exceptions have several layered memory costs that compound significantly under high load or in exception-heavy code paths.

1. Stack trace allocation (dominant cost): `fillInStackTrace()` is called in the Throwable constructor. It uses a native JVM API to walk the entire call stack and allocates a `StackTraceElement[]` — one element per frame. Each `StackTraceElement` holds four String references (`declaringClass`, `methodName`, `fileName`, a boxed int for line number). For a typical web framework stack of 80 frames: 80 `StackTraceElement` objects + ~320 String references (many interned, but not all) + the array itself ≈ 40-100KB of object graph per exception.
2. The exception object itself: Throwable fields: `detailMessage` (String ref), `cause` (Throwable ref), `stackTrace` (`StackTraceElement[]`), `suppressedExceptions` (List, lazily initialized via a sentinel to avoid allocation for simple exceptions). Total object overhead: ~64-80 bytes plus the trace.
3. GC pressure: At 100 exceptions/second with 80KB traces: 8MB/second of short-lived objects entering the young generation. This triggers frequent minor GCs. If some exceptions survive (e.g., stored in error lists), they promote to old generation, contributing to major GC pressure.
4. String intern interaction: Class names and method names in `StackTraceElement` are usually interned — they exist once in the String pool. File names and line numbers may be new String objects per `StackTraceElement`. In practice, the `StackTraceElement` objects themselves and the array are the main allocation.

Memory optimization techniques:

- Override `fillInStackTrace()`: `public synchronized Throwable fillInStackTrace() { return this; }` — creates no trace, reducing exception creation to a single small object. Use for signal exceptions where location is irrelevant.
- `setStackTrace(new StackTraceElement[0])` — clear a trace after catch if it will be logged and discarded.
- Pre-allocate static final exception instances for purely signaling use (e.g., a sentinel EOF exception in a parser) — zero allocation per throw.
- Monitor with JFR (Java Flight Recorder) or `async-profiler`: allocation profiling identifies exception-heavy code paths. Exception events in JFR capture exception type and frequency with minimal overhead.

Q10. Deep dive into the Throwable class internals.

Throwable is the root of Java's exception system. Understanding its internals illuminates how every exception in Java works. Core fields (as of OpenJDK 17): `private String detailMessage` — the human-readable error text passed to the constructor. `private Throwable cause` — the wrapped exception for chaining. Default is 'this' (sentinel meaning 'cause not yet set'), set to null by no-arg constructor (no cause), or to an actual Throwable via the `cause` constructor or `initCause()`. `private StackTraceElement[] stackTrace` — captured by `fillInStackTrace()`. Can be null (native exceptions) or an empty array (suppressed trace). `private List suppressedExceptions` — initialized lazily. The sentinel

`SUPPRESSED_SENTINEL` (`Collections.emptyList()`) is used to avoid allocating a `List` for the common case where no exceptions are suppressed. `private transient Object backtrace` — an opaque reference to the JVM's internal representation of the stack trace (JVM-specific, not accessible from Java). `stackTrace` is materialized from `backtrace` on first call to `getStackTrace()`. Key methods: `fillInStackTrace()`: synchronized native method. Populates `backtrace` by walking the JVM's frame list. Called automatically in the `Throwable` constructor. Overriding to return `'this'` disables capture. `getStackTrace()`: Returns a clone of `stackTrace` (or materializes it from `backtrace` on first access). Cloned to prevent external modification. `printStackTrace(PrintWriter s)`: Writes `className + ": " + message`, then each `StackTraceElement` as `" at class.method(file:line)"`, then recursively prints causes (prefixed "Caused by:") and suppressed exceptions (prefixed " Suppressed: "). `initCause(Throwable cause)`: Sets the cause field. Can only be called once; throws `IllegalStateException` if called again. Required for exceptions predating the cause-constructor pattern (pre-Java 1.4 exceptions). `addSuppressed(Throwable exception)`: Lazily initializes `suppressedExceptions` and adds the exception. Cannot add null or `'this'`. Thread-safe via `synchronized`. `getSuppressed()`: Returns a copy of `suppressedExceptions` as `Throwable[]` (or an empty array if none). Serialization: `Throwable` implements `Serializable`. Both `stackTrace` (`StackTraceElement[]` is serializable) and `cause` are serialized. This enables exceptions to cross JVM boundaries via RMI, object serialization, or distributed logs. The `backtrace` field is transient (JVM-internal, not serializable); `getStackTrace()` is called before serialization to materialize `stackTrace`. Thread safety: `stackTrace` and `backtrace` accesses are not synchronized (`getStackTrace()` is). `fillInStackTrace()` is synchronized. `addSuppressed()` is synchronized. Concurrent throws from the same `Throwable` instance are not a supported pattern. Best practices derived from internals: Always provide constructors with `(String message)`, `(String message, Throwable cause)`, `(Throwable cause)`, and no-arg for maximum compatibility. Never hold strong references to exceptions in long-lived structures (prevents GC of everything in the stack trace Strings). Override `fillInStackTrace()` for high-frequency signal exceptions to eliminate the dominant memory and CPU cost.

■ Practical Questions & Programs

■ EASY PRACTICE — 20 Programs

■ Practice 1: Divide Two Numbers — Handle ArithmeticException

```
1 ■ public class P01_Divide {
2 ■     public static void main(String[] args) {
3 ■         int a = 10, b = 0;
4 ■         try {
5 ■             int result = a / b;
6 ■             System.out.println("Result: " + result);
7 ■         } catch (ArithmeticException e) {
8 ■             System.out.println("Cannot divide by zero: " + e.getMessage());
9 ■         } finally {
10 ■             System.out.println("Division attempt finished.");
11 ■         }
12 ■     }
13 ■ }
```

■ Practice 2: Handle ArithmeticException with Method

```
1 ■ public class P02_ArithmeticEx {
2 ■     static int divide(int x, int y) {
3 ■         if (y == 0) throw new ArithmeticException("Divisor is zero");
4 ■         return x / y;
5 ■     }
6 ■     public static void main(String[] args) {
7 ■         int[][] pairs = {{20, 4}, {10, 0}, {15, 3}};
8 ■         for (int[] p : pairs) {
9 ■             try {
10 ■                 System.out.printf("%d / %d = %d\n", p[0], p[1], divide(p[0], p[1]));
11 ■             } catch (ArithmeticException e) {
12 ■                 System.out.println("Error: " + e.getMessage());
13 ■             }
14 ■         }
15 ■     }
16 ■ }
```

■ Practice 3: Handle NullPointerException

```
1 ■ public class P03_NullPointer {
2 ■     static int stringLength(String s) {
3 ■         if (s == null) throw new NullPointerException("Input string is null");
4 ■         return s.length();
5 ■     }
6 ■     public static void main(String[] args) {
7 ■         String[] inputs = {"Hello", null, "Java"};
```



```

8 ■ for (String s : inputs) {
9 ■ try {
10 ■ System.out.println("Length of " + s + ": " + stringLength(s));
11 ■ } catch (NullPointerException e) {
12 ■ System.out.println("Caught NPE: " + e.getMessage());
13 ■ }
14 ■ }
15 ■ }
16 ■ }

```

■ Practice 4: Handle ArrayIndexOutOfBoundsException

```

1 ■ public class P04_ArrayBounds {
2 ■ public static void main(String[] args) {
3 ■ int[] data = {10, 20, 30};
4 ■ int[] indices = {0, 1, 2, 5, -1};
5 ■ for (int idx : indices) {
6 ■ try {
7 ■ System.out.println("data[" + idx + "] = " + data[idx]);
8 ■ } catch (ArrayIndexOutOfBoundsException e) {
9 ■ System.out.println("Bad index " + idx + ": " + e.getMessage());
10 ■ }
11 ■ }
12 ■ }
13 ■ }

```

■ Practice 5: Simple try-catch Example

```

1 ■ public class P05_SimpleTryCatch {
2 ■ public static void main(String[] args) {
3 ■ System.out.println("Program start");
4 ■ try {
5 ■ int[] arr = new int[3];
6 ■ arr[10] = 42; // triggers exception
7 ■ System.out.println("This line is unreachable");
8 ■ } catch (ArrayIndexOutOfBoundsException e) {
9 ■ System.out.println("Caught: " + e.getClass().getSimpleName());
10 ■ System.out.println("Message: " + e.getMessage());
11 ■ }
12 ■ System.out.println("Program continues normally after catch.");
13 ■ }
14 ■ }

```

■ Practice 6: Use finally Block

```

1 ■ public class P06_Finally {
2 ■ static void riskyMethod(boolean shouldFail) {
3 ■ try {
4 ■ System.out.println(" try block executing");
5 ■ if (shouldFail) throw new RuntimeException("Forced failure");

```

```
6 ■ System.out.println(" try block succeeded");
7 ■ } catch (RuntimeException e) {
8 ■ System.out.println(" catch block: " + e.getMessage());
9 ■ } finally {
10 ■ System.out.println(" finally block ALWAYS runs");
11 ■ }
12 ■ }
13 ■ public static void main(String[] args) {
14 ■ System.out.println("--- Success scenario ---");
15 ■ riskyMethod(false);
16 ■ System.out.println("--- Failure scenario ---");
17 ■ riskyMethod(true);
18 ■ }
19 ■ }
```

■ Practice 7: Multiple catch Blocks

```
1 ■ public class P07_MultipleCatch {
2 ■ static void process(int choice) {
3 ■ switch (choice) {
4 ■ case 1: throw new NullPointerException("null ref");
5 ■ case 2: throw new ArrayIndexOutOfBoundsException("bad idx");
6 ■ case 3: throw new NumberFormatException("bad number");
7 ■ default: System.out.println("Choice " + choice + " – no exception");
8 ■ }
9 ■ }
10 ■ public static void main(String[] args) {
11 ■ for (int i = 0; i <= 4; i++) {
12 ■ try {
13 ■ process(i);
14 ■ } catch (NullPointerException e) {
15 ■ System.out.println("NPE: " + e.getMessage());
16 ■ } catch (ArrayIndexOutOfBoundsException e) {
17 ■ System.out.println("AI00B: " + e.getMessage());
18 ■ } catch (NumberFormatException e) {
19 ■ System.out.println("NFE: " + e.getMessage());
20 ■ }
21 ■ }
22 ■ }
23 ■ }
```

■ Practice 8: Nested try-catch

```
1 ■ public class P08_NestedTry {
2 ■ public static void main(String[] args) {
3 ■ try {
4 ■ System.out.println("Outer try start");
5 ■ try {
6 ■ System.out.println(" Inner try: attempting division");
7 ■ int r = 10 / 0;
```

```

8 ■ } catch (ArithmeticException e) {
9 ■ System.out.println(" Inner catch: " + e.getMessage());
10 ■ System.out.println(" Rethrowing as RuntimeException...");
11 ■ throw new RuntimeException("Wrapped: " + e.getMessage(), e);
12 ■ }
13 ■ } catch (RuntimeException e) {
14 ■ System.out.println("Outer catch: " + e.getMessage());
15 ■ System.out.println("Cause: " + e.getCause().getClass().getSimpleName());
16 ■ } finally {
17 ■ System.out.println("Outer finally");
18 ■ }
19 ■ }
20 ■ }

```

■ Practice 9: Use throw Keyword

```

1 ■ public class P09_Throw {
2 ■ static void validateAge(int age) {
3 ■ if (age < 0) throw new IllegalArgumentException("Age cannot be negative: " + age);
4 ■ if (age < 18) throw new IllegalArgumentException("Must be 18 or older. Got: " + age);
5 ■ if (age > 120) throw new IllegalArgumentException("Unrealistic age: " + age);
6 ■ }
7 ■ public static void main(String[] args) {
8 ■ int[] ages = {25, -5, 15, 130, 18};
9 ■ for (int age : ages) {
10 ■ try {
11 ■ validateAge(age);
12 ■ System.out.println("Age " + age + " is valid.");
13 ■ } catch (IllegalArgumentException e) {
14 ■ System.out.println("Invalid: " + e.getMessage());
15 ■ }
16 ■ }
17 ■ }
18 ■ }

```

■ Practice 10: Use throws Keyword

```

1 ■ import java.io.*;
2 ■ public class P10_Throws {
3 ■ // declares that it may propagate IOException
4 ■ static String readFirstLine(String path) throws IOException {
5 ■ try (BufferedReader br = new BufferedReader(new FileReader(path))) {
6 ■ return br.readLine();
7 ■ }
8 ■ }
9 ■ // caller must handle or re-declare
10 ■ public static void main(String[] args) {
11 ■ try {
12 ■ String line = readFirstLine("sample.txt");
13 ■ System.out.println("First line: " + line);

```

```

14 ■ } catch (FileNotFoundException e) {
15 ■     System.out.println("File not found: " + e.getMessage());
16 ■ } catch (IOException e) {
17 ■     System.out.println("IO error: " + e.getMessage());
18 ■ }
19 ■ }
20 ■ }

```

■ Practice 11: Handle FileNotFoundException

```

1 ■ import java.io.*;
2 ■ public class P11_FileNotFound {
3 ■     public static void main(String[] args) {
4 ■         String[] files = {"existing.txt", "missing.txt", "also_missing.txt"};
5 ■         for (String file : files) {
6 ■             try {
7 ■                 FileReader fr = new FileReader(file);
8 ■                 System.out.println("Opened: " + file);
9 ■                 fr.close();
10 ■            } catch (FileNotFoundException e) {
11 ■                System.out.println("Not found: " + file
12 ■                    + " - " + e.getMessage());
13 ■            } catch (IOException e) {
14 ■                System.out.println("IO error: " + e.getMessage());
15 ■            }
16 ■        }
17 ■    }
18 ■ }

```

■ Practice 12: Print Stack Trace

```

1 ■ public class P12_StackTrace {
2 ■     static void methodC() { throw new RuntimeException("Error in C"); }
3 ■     static void methodB() { methodC(); }
4 ■     static void methodA() { methodB(); }
5 ■     public static void main(String[] args) {
6 ■         try {
7 ■             methodA();
8 ■         } catch (RuntimeException e) {
9 ■             System.out.println("=== getMessage() ===");
10 ■             System.out.println(e.getMessage());
11 ■             System.out.println("=== toString() ===");
12 ■             System.out.println(e.toString());
13 ■             System.out.println("=== printStackTrace() ===");
14 ■             e.printStackTrace();
15 ■             System.out.println("=== Manual stack frames ===");
16 ■             for (StackTraceElement f : e.getStackTrace()) {
17 ■                 System.out.println(" at " + f);
18 ■             }
19 ■         }

```

```
20 ■ }  
21 ■ }
```

■ Practice 13: Print Exception Message

```
1 ■ public class P13_Message {  
2 ■     public static void main(String[] args) {  
3 ■         try {  
4 ■             int result = Integer.parseInt("not-a-number");  
5 ■         } catch (NumberFormatException e) {  
6 ■             System.out.println("getMessage() : " + e.getMessage());  
7 ■             System.out.println("toString() : " + e.toString());  
8 ■             System.out.println("getClass() : " + e.getClass().getName());  
9 ■             System.out.println("getSimpleName: " + e.getClass().getSimpleName());  
10 ■         }  
11 ■     }  
12 ■ }
```

■ Practice 14: Demonstrate Unchecked Exception

```
1 ■ public class P14_Unchecked {  
2 ■     // No try-catch required – compiler does NOT enforce handling  
3 ■     static int divideUnchecked(int a, int b) {  
4 ■         return a / b; // throws ArithmeticException (unchecked)  
5 ■     }  
6 ■     public static void main(String[] args) {  
7 ■         // Safe call  
8 ■         System.out.println(divideUnchecked(10, 2));  
9 ■         // Unsafe call – no compile-time warning; crashes at runtime  
10 ■        System.out.println(divideUnchecked(10, 0));  
11 ■    }  
12 ■ }
```

■ Practice 15: Demonstrate Checked Exception

```
1 ■ import java.io.*;  
2 ■ public class P15_Checked {  
3 ■     // COMPILE ERROR if we don't handle or declare IOException  
4 ■     static void writeToFile(String content) throws IOException {  
5 ■         FileWriter fw = new FileWriter("output.txt");  
6 ■         fw.write(content);  
7 ■         fw.close();  
8 ■         System.out.println("Written: " + content);  
9 ■     }  
10 ■     public static void main(String[] args) {  
11 ■         try {  
12 ■             writeToFile("Hello from Java!");  
13 ■         } catch (IOException e) {  
14 ■             System.out.println("Write failed: " + e.getMessage());  
15 ■         }  
16 ■     }  
17 ■ }
```

```
16 ■ }
17 ■ }
```

■ Practice 16: Use try-with-resources

```
1 ■ import java.io.*;
2 ■ public class P16_TryWithResources {
3 ■     public static void main(String[] args) {
4 ■         // BufferedReader is auto-closed when try block exits
5 ■         String path = "data.txt";
6 ■         try (BufferedReader br = new BufferedReader(new FileReader(path))) {
7 ■             String line;
8 ■             int lineNo = 1;
9 ■             while ((line = br.readLine()) != null) {
10 ■                 System.out.printf("%3d: %s\n", lineNo++, line);
11 ■             }
12 ■         } catch (FileNotFoundException e) {
13 ■             System.out.println("File not found: " + path);
14 ■         } catch (IOException e) {
15 ■             System.out.println("Read error: " + e.getMessage());
16 ■         }
17 ■         // br is guaranteed to be closed here – no finally needed
18 ■         System.out.println("Resource auto-closed.");
19 ■     }
20 ■ }
```

■ Practice 17: Close Resource Manually (try-finally)

```
1 ■ import java.io.*;
2 ■ public class P17_ManualClose {
3 ■     public static void main(String[] args) {
4 ■         BufferedReader br = null;
5 ■         try {
6 ■             br = new BufferedReader(new FileReader("data.txt"));
7 ■             System.out.println(br.readLine());
8 ■         } catch (FileNotFoundException e) {
9 ■             System.out.println("Not found: " + e.getMessage());
10 ■         } catch (IOException e) {
11 ■             System.out.println("Read error: " + e.getMessage());
12 ■         } finally {
13 ■             // Must close manually – more verbose than try-with-resources
14 ■             if (br != null) {
15 ■                 try {
16 ■                     br.close();
17 ■                     System.out.println("Resource closed in finally.");
18 ■                 } catch (IOException e) {
19 ■                     System.out.println("Close failed: " + e.getMessage());
20 ■                 }
21 ■             }
22 ■         }
```

```
23 ■ }
24 ■ }
```

■ Practice 18: Exception Propagation Example

```
1 ■ public class P18_Propagation {
2 ■     static void level3() throws Exception {
3 ■         System.out.println(" level3: about to throw");
4 ■         throw new Exception("Origin: level3");
5 ■     }
6 ■     static void level2() throws Exception {
7 ■         System.out.println(" level2: calling level3");
8 ■         level3(); // propagates – no catch here
9 ■         System.out.println(" level2: this is unreachable");
10 ■    }
11 ■    static void level1() throws Exception {
12 ■        System.out.println("level1: calling level2");
13 ■        level2(); // propagates – no catch here
14 ■    }
15 ■    public static void main(String[] args) {
16 ■        System.out.println("main: calling level1");
17 ■        try {
18 ■            level1();
19 ■        } catch (Exception e) {
20 ■            System.out.println("main: caught – " + e.getMessage());
21 ■            e.printStackTrace();
22 ■        }
23 ■    }
24 ■ }
```

■ Practice 19: Basic Custom Exception

```
1 ■ // Checked custom exception
2 ■ class AppException extends Exception {
3 ■     private final int code;
4 ■     public AppException(String msg, int code) { super(msg); this.code = code; }
5 ■     public int getCode() { return code; }
6 ■ }
7 ■
8 ■ public class P19_CustomEx {
9 ■     static void process(int value) throws AppException {
10 ■         if (value < 0)
11 ■             throw new AppException("Negative value rejected: " + value, 400);
12 ■         if (value > 100)
13 ■             throw new AppException("Value exceeds max (100): " + value, 422);
14 ■         System.out.println("Processed successfully: " + value);
15 ■     }
16 ■     public static void main(String[] args) {
17 ■         int[] vals = {50, -10, 150, 75};
18 ■         for (int v : vals) {
```



```

19 ■ try {
20 ■ process(v);
21 ■ } catch (AppException e) {
22 ■ System.out.printf("AppException [%d]: %s\n", e.getCode(), e.getMessage());
23 ■ }
24 ■ }
25 ■ }
26 ■ }

```

■ Practice 20: Input Validation with Exception

```

1 ■ import java.util.regex.*;
2 ■ public class P20_InputValidation {
3 ■ static void validateEmail(String email) {
4 ■ if (email == null || email.isEmpty())
5 ■ throw new IllegalArgumentException("Email cannot be blank");
6 ■ if (!email.matches("^[\\w.-]+@[\\w.-]+\\. [a-zA-Z]{2,}$"))
7 ■ throw new IllegalArgumentException("Invalid email format: " + email);
8 ■ }
9 ■ static int parsePositiveInt(String s) {
10 ■ try {
11 ■ int n = Integer.parseInt(s);
12 ■ if (n <= 0) throw new IllegalArgumentException("Must be positive, got: " + n);
13 ■ return n;
14 ■ } catch (NumberFormatException e) {
15 ■ throw new IllegalArgumentException("Not a number: " + s, e);
16 ■ }
17 ■ }
18 ■ public static void main(String[] args) {
19 ■ String[] emails = {"user@example.com", "bad-email", "", "a@b.co"};
20 ■ for (String e : emails) {
21 ■ try { validateEmail(e); System.out.println("Valid: " + e); }
22 ■ catch (IllegalArgumentException ex) { System.out.println("Invalid: " + ex.getMessage()); }
23 ■ }
24 ■ String[] nums = {"42", "-3", "abc", "100"};
25 ■ for (String n : nums) {
26 ■ try { System.out.println("Parsed: " + parsePositiveInt(n)); }
27 ■ catch (IllegalArgumentException ex) { System.out.println("Error: " + ex.getMessage()); }
28 ■ }
29 ■ }
30 ■ }

```

■■ MEDIUM PRACTICE — 20 Programs

■ Practice 1: Create Custom Exception Class

```

1 ■ // Full-featured custom exception hierarchy
2 ■ class DomainException extends RuntimeException {

```

```
3 ■ private final String errorCode;
4 ■ public DomainException(String msg, String code) { super(msg); this.errorCode = code; }
5 ■ public DomainException(String msg, String code, Throwable cause) {
6 ■     super(msg, cause); this.errorCode = code;
7 ■ }
8 ■ public String getErrorCode() { return errorCode; }
9 ■ }
10 ■
11 ■ class ValidationException extends DomainException {
12 ■     private final String field;
13 ■     public ValidationException(String field, String reason) {
14 ■         super("Validation failed for '" + field + "': " + reason, "VALIDATION_ERR");
15 ■         this.field = field;
16 ■     }
17 ■     public String getField() { return field; }
18 ■ }
19 ■
20 ■ class ResourceNotFoundException extends DomainException {
21 ■     public ResourceNotFoundException(String resource, Object id) {
22 ■         super(resource + " not found with id=" + id, "NOT_FOUND");
23 ■     }
24 ■ }
25 ■
26 ■ public class M01_CustomExceptionClass {
27 ■     static void validateName(String name) {
28 ■         if (name == null || name.isBlank())
29 ■             throw new ValidationException("name", "cannot be blank");
30 ■         if (name.length() < 2)
31 ■             throw new ValidationException("name", "must be at least 2 chars");
32 ■     }
33 ■     static String findUser(int id) {
34 ■         if (id != 1) throw new ResourceNotFoundException("User", id);
35 ■         return "Alice";
36 ■     }
37 ■     public static void main(String[] args) {
38 ■         String[] names = {"Alice", "", "X"};
39 ■         for (String n : names) {
40 ■             try { validateName(n); System.out.println("Name OK: " + n); }
41 ■             catch (ValidationException e) {
42 ■                 System.out.printf("[%s] field=%s msg=%s\n",
43 ■                     e.getErrorCode(), e.getField(), e.getMessage());
44 ■             }
45 ■         }
46 ■         for (int id : new int[]{1, 2, 99}) {
47 ■             try { System.out.println("Found: " + findUser(id)); }
48 ■             catch (ResourceNotFoundException e) {
49 ■                 System.out.printf("[%s] %s\n", e.getErrorCode(), e.getMessage());
50 ■             }
51 ■         }
52 ■     }
```

```
53 ■ }
```

■ Practice 2: Age Validation Program

```
1 ■ class InvalidAgeException extends Exception {
2 ■     private final int providedAge;
3 ■     public InvalidAgeException(int age, String reason) {
4 ■         super(String.format("Invalid age %d: %s", age, reason));
5 ■         this.providedAge = age;
6 ■     }
7 ■     public int getProvidedAge() { return providedAge; }
8 ■ }
9 ■
10 ■ public class M02_AgeValidation {
11 ■     static final int MIN_WORKING_AGE = 18;
12 ■     static final int MAX_WORKING_AGE = 65;
13 ■     static final int MAX_HUMAN_AGE = 130;
14 ■
15 ■     static void validateAge(int age) throws InvalidAgeException {
16 ■         if (age < 0) throw new InvalidAgeException(age, "age cannot be negative");
17 ■         if (age > MAX_HUMAN_AGE) throw new InvalidAgeException(age, "exceeds maximum human age");
18 ■         if (age < MIN_WORKING_AGE) throw new InvalidAgeException(age, "below working age (18)");
19 ■         if (age > MAX_WORKING_AGE) throw new InvalidAgeException(age, "above retirement age
(65)");
20 ■         System.out.printf("Age %d is valid for employment.%n", age);
21 ■     }
22 ■
23 ■     public static void main(String[] args) {
24 ■         int[] ages = {25, -1, 200, 17, 66, 18, 65};
25 ■         for (int age : ages) {
26 ■             try {
27 ■                 validateAge(age);
28 ■             } catch (InvalidAgeException e) {
29 ■                 System.out.println("REJECTED: " + e.getMessage()
30 ■ + " (provided: " + e.getProvidedAge() + ")");
31 ■             }
32 ■         }
33 ■     }
34 ■ }
```

■ Practice 3: Bank Withdrawal Exception

```
1 ■ class InsufficientFundsException extends Exception {
2 ■     private final double balance, amount;
3 ■     public InsufficientFundsException(double balance, double amount) {
4 ■         super(String.format("Insufficient funds. Balance: %.2f, Requested: %.2f", balance,
amount));
5 ■         this.balance = balance; this.amount = amount;
6 ■     }
7 ■     public double getShortfall() { return amount - balance; }
```

```

8 ■ }
9 ■
10 ■ class BankAccount {
11 ■     private final String owner;
12 ■     private double balance;
13 ■     BankAccount(String owner, double balance) { this.owner = owner; this.balance = balance; }
14 ■     void deposit(double amount) {
15 ■         if (amount <= 0) throw new IllegalArgumentException("Deposit must be positive");
16 ■         balance += amount;
17 ■         System.out.printf("[%s] Deposited %.2f → Balance: %.2f%n", owner, amount, balance);
18 ■     }
19 ■     void withdraw(double amount) throws InsufficientFundsException {
20 ■         if (amount <= 0) throw new IllegalArgumentException("Withdrawal must be positive");
21 ■         if (amount > balance) throw new InsufficientFundsException(balance, amount);
22 ■         balance -= amount;
23 ■         System.out.printf("[%s] Withdrew %.2f → Balance: %.2f%n", owner, amount, balance);
24 ■     }
25 ■     double getBalance() { return balance; }
26 ■ }
27 ■
28 ■ public class M03_BankWithdrawal {
29 ■     public static void main(String[] args) {
30 ■         BankAccount acc = new BankAccount("Alice", 1000.0);
31 ■         double[] amounts = {200, 500, 600, 100};
32 ■         for (double amt : amounts) {
33 ■             try {
34 ■                 acc.withdraw(amt);
35 ■             } catch (InsufficientFundsException e) {
36 ■                 System.out.printf("FAILED: %s (Shortfall: %.2f)%n",
37 ■                     e.getMessage(), e.getShortfall());
38 ■             }
39 ■         }
40 ■     }
41 ■ }

```

■ Practice 4: File Reading Using throws

```

1 ■ import java.io.*;
2 ■ import java.util.*;
3 ■
4 ■ public class M04_FileReading {
5 ■     static List<String> readAllLines(String filename) throws IOException {
6 ■         List<String> lines = new ArrayList<>();
7 ■         try (BufferedReader br = new BufferedReader(new FileReader(filename))) {
8 ■             String line;
9 ■             while ((line = br.readLine()) != null) lines.add(line);
10 ■        }
11 ■        return lines;
12 ■    }
13 ■    static Map<String, Integer> wordCount(String filename) throws IOException {

```

```

14 ■ Map<String, Integer> counts = new TreeMap<>();
15 ■ for (String line : readAllLines(filename)) {
16 ■     for (String word : line.split("\\W+")) {
17 ■         if (!word.isEmpty())
18 ■             counts.merge(word.toLowerCase(), 1, Integer::sum);
19 ■     }
20 ■ }
21 ■ return counts;
22 ■ }
23 ■ public static void main(String[] args) {
24 ■     try {
25 ■         List<String> lines = readAllLines("document.txt");
26 ■         System.out.println("Lines: " + lines.size());
27 ■         lines.forEach(l -> System.out.println(" " + l));
28 ■         Map<String, Integer> wc = wordCount("document.txt");
29 ■         System.out.println("Word counts: " + wc);
30 ■     } catch (FileNotFoundException e) {
31 ■         System.out.println("File not found: " + e.getMessage());
32 ■     } catch (IOException e) {
33 ■         System.out.println("IO error: " + e.getMessage());
34 ■     }
35 ■ }
36 ■ }

```

■ Practice 5: Rethrow Exception

```

1 ■ import java.io.*;
2 ■ import java.util.logging.*;
3 ■
4 ■ public class M05_Rethrow {
5 ■     static final Logger LOG = Logger.getLogger(M05_Rethrow.class.getName());
6 ■
7 ■     static String loadConfig(String path) throws IOException {
8 ■         try (BufferedReader br = new BufferedReader(new FileReader(path))) {
9 ■             return br.readLine();
10 ■        } catch (FileNotFoundException e) {
11 ■            LOG.warning("Config missing at: " + path);
12 ■            throw e; // rethrow same exception after logging
13 ■        }
14 ■    }
15 ■
16 ■     static void initializeApp() {
17 ■         try {
18 ■             String cfg = loadConfig("app.properties");
19 ■             System.out.println("Config loaded: " + cfg);
20 ■        } catch (IOException e) {
21 ■            // Wrap in unchecked for higher layers
22 ■            throw new RuntimeException("App initialization failed", e);
23 ■        }
24 ■    }

```

```

25 ■
26 ■ public static void main(String[] args) {
27 ■     try {
28 ■         initializeApp();
29 ■     } catch (RuntimeException e) {
30 ■         System.out.println("FATAL: " + e.getMessage());
31 ■         System.out.println("Root cause: " + e.getCause().getClass().getSimpleName()
32 ■ + " - " + e.getCause().getMessage());
33 ■     }
34 ■ }
35 ■ }

```

■ Practice 6: Multi-catch Example

```

1 ■ public class M06_MultiCatch {
2 ■     static void parse(String input) throws Exception {
3 ■         if (input == null) throw new NullPointerException("Input is null");
4 ■         if (input.isEmpty()) throw new IllegalArgumentException("Input is empty");
5 ■         try {
6 ■             int val = Integer.parseInt(input);
7 ■             if (val < 0) throw new ArithmeticException("Negative value: " + val);
8 ■             System.out.println("Parsed: " + val);
9 ■         } catch (NumberFormatException e) {
10 ■             throw new NumberFormatException("Not an integer: " + input);
11 ■         }
12 ■     }
13 ■     public static void main(String[] args) {
14 ■         String[] inputs = {"42", null, "", "abc", "-5", "100"};
15 ■         for (String s : inputs) {
16 ■             try {
17 ■                 parse(s);
18 ■             } catch (NullPointerException | IllegalArgumentException e) {
19 ■                 // Multi-catch: same handling for both types
20 ■                 System.out.println("[INPUT_ERR] " + e.getClass().getSimpleName()
21 ■ + ": " + e.getMessage());
22 ■             } catch (NumberFormatException | ArithmeticException e) {
23 ■                 // Multi-catch: same handling for numeric errors
24 ■                 System.out.println("[NUMERIC_ERR] " + e.getClass().getSimpleName()
25 ■ + ": " + e.getMessage());
26 ■             } catch (Exception e) {
27 ■                 System.out.println("[OTHER] " + e.getMessage());
28 ■             }
29 ■         }
30 ■     }
31 ■ }

```

■ Practice 7: Chained Exception Example

```

1 ■ class DataAccessException extends RuntimeException {
2 ■     public DataAccessException(String msg, Throwable cause) { super(msg, cause); }

```

```

3 ■ }
4 ■ class ServiceException extends RuntimeException {
5 ■     public ServiceException(String msg, Throwable cause) { super(msg, cause); }
6 ■ }
7 ■
8 ■ public class M07_ChainedException {
9 ■     static void queryDB(String sql) throws DataAccessException {
10 ■         try {
11 ■             if (sql.contains("DROP")) throw new java.sql.SQLException("Dangerous query blocked");
12 ■             System.out.println("Query OK: " + sql);
13 ■         } catch (java.sql.SQLException e) {
14 ■             throw new DataAccessException("DB layer failure for: " + sql, e);
15 ■         }
16 ■     }
17 ■     static String getUser_name(int userId) {
18 ■         try {
19 ■             queryDB("SELECT name FROM users WHERE id=" + userId);
20 ■             return "Alice";
21 ■         } catch (DataAccessException e) {
22 ■             throw new ServiceException("Cannot fetch user " + userId, e);
23 ■         }
24 ■     }
25 ■     public static void main(String[] args) {
26 ■         try {
27 ■             System.out.println(getUser_name(1));
28 ■             System.out.println(getUser_name(2)); // triggers chain
29 ■         } catch (ServiceException e) {
30 ■             System.out.println("Service: " + e.getMessage());
31 ■             System.out.println(" Caused by: " + e.getCause().getMessage());
32 ■             System.out.println(" Root cause: " + e.getCause().getCause().getMessage());
33 ■             System.out.println("--- Full stack trace ---");
34 ■             e.printStackTrace();
35 ■         }
36 ■     }
37 ■ }

```

■ Practice 8: Logging Exception Program

```

1 ■ import java.util.logging.*;
2 ■ import java.io.*;
3 ■
4 ■ public class M08_LoggingException {
5 ■     private static final Logger LOGGER =
6 ■         Logger.getLogger(M08_LoggingException.class.getName());
7 ■
8 ■     static {
9 ■         LOGGER.setLevel(Level.ALL);
10 ■         ConsoleHandler ch = new ConsoleHandler();
11 ■         ch.setLevel(Level.ALL);
12 ■         ch.setFormatter(new SimpleFormatter());

```

```

12 ■ LOGGER.addHandler(ch);
13 ■ LOGGER.setUseParentHandlers(false);
14 ■ }
15 ■
16 ■ static double safeDivide(double a, double b) {
17 ■ try {
18 ■ if (b == 0) throw new ArithmeticException("Division by zero");
19 ■ double result = a / b;
20 ■ LOGGER.info(String.format("Computed %.2f / %.2f = %.4f", a, b, result));
21 ■ return result;
22 ■ } catch (ArithmeticException e) {
23 ■ LOGGER.log(Level.WARNING, "Math error in safeDivide()", e);
24 ■ return Double.NaN;
25 ■ }
26 ■ }
27 ■ static String readFile(String path) throws IOException {
28 ■ LOGGER.info("Reading file: " + path);
29 ■ try (BufferedReader br = new BufferedReader(new FileReader(path))) {
30 ■ return br.readLine();
31 ■ } catch (IOException e) {
32 ■ LOGGER.log(Level.SEVERE, "Failed to read: " + path, e);
33 ■ throw e;
34 ■ }
35 ■ }
36 ■ public static void main(String[] args) {
37 ■ System.out.println(safeDivide(10, 3));
38 ■ System.out.println(safeDivide(10, 0));
39 ■ try { readFile("missing.txt"); }
40 ■ catch (IOException e) { LOGGER.severe("Unrecoverable IO: " + e.getMessage()); }
41 ■ }
42 ■ }

```

■ Practice 9: Nested try with Multiple Cases

```

1 ■ public class M09_NestedMultiple {
2 ■ static int parseAndDivide(String aStr, String bStr) throws Exception {
3 ■ int a, b;
4 ■ try {
5 ■ a = Integer.parseInt(aStr);
6 ■ } catch (NumberFormatException e) {
7 ■ throw new Exception("Bad dividend '" + aStr + "'", e);
8 ■ }
9 ■ try {
10 ■ b = Integer.parseInt(bStr);
11 ■ if (b == 0) throw new ArithmeticException("Divisor is zero");
12 ■ } catch (NumberFormatException e) {
13 ■ throw new Exception("Bad divisor '" + bStr + "'", e);
14 ■ }
15 ■ try {
16 ■ return a / b;

```



```

17 ■ } catch (ArithmeticException e) {
18 ■     throw new Exception("Division failed: " + a + "/" + b, e);
19 ■ }
20 ■ }
21 ■ public static void main(String[] args) {
22 ■     String[][] tests = {
23 ■         {"10", "2"}, {"abc", "2"}, {"10", "xyz"}, {"10", "0"}, {"100", "4"}
24 ■     };
25 ■     for (String[] t : tests) {
26 ■         try {
27 ■             System.out.printf("%s / %s = %d%n", t[0], t[1], parseAndDivide(t[0], t[1]));
28 ■         } catch (Exception e) {
29 ■             System.out.println("Error: " + e.getMessage());
30 ■             if (e.getCause() != null)
31 ■                 System.out.println(" Cause: " + e.getCause().getMessage());
32 ■         }
33 ■     }
34 ■ }
35 ■ }

```

■ Practice 10: Resource Handling System (AutoCloseable)

```

1 ■ // Multiple AutoCloseable resources
2 ■ class DBConnection implements AutoCloseable {
3 ■     private final String url;
4 ■     private boolean open;
5 ■     DBConnection(String url) { this.url = url; open = true; System.out.println("DB opened: " + url); }
6 ■     String query(String sql) {
7 ■         if (!open) throw new IllegalStateException("Connection already closed");
8 ■         return "Rows for [" + sql + "]";
9 ■     }
10 ■     @Override public void close() {
11 ■         open = false;
12 ■         System.out.println("DB closed: " + url);
13 ■     }
14 ■ }
15 ■
16 ■ class CsvWriter implements AutoCloseable {
17 ■     private final String path;
18 ■     CsvWriter(String path) { this.path = path; System.out.println("CSV writer opened: " + path); }
19 ■     void write(String line) { System.out.println("CSV write: " + line); }
20 ■     @Override public void close() { System.out.println("CSV writer closed: " + path); }
21 ■ }
22 ■
23 ■ public class M10_ResourceHandling {
24 ■     public static void main(String[] args) {
25 ■         // Both resources auto-closed in reverse order (CsvWriter first, then DBConnection)
26 ■         try (DBConnection db = new DBConnection("jdbc:mysql://localhost/mydb");

```

```

27 ■ CsvWriter csv = new CsvWriter("output.csv") {
28 ■ String result = db.query("SELECT * FROM products");
29 ■ csv.write(result);
30 ■ csv.write("Done");
31 ■ } catch (Exception e) {
32 ■ System.out.println("Error: " + e.getMessage());
33 ■ }
34 ■ System.out.println("Both resources confirmed closed.");
35 ■ }
36 ■ }

```

■ Practice 11: Exception in Constructor

```

1 ■ import java.io.*;
2 ■
3 ■ class Config implements AutoCloseable {
4 ■ private final BufferedReader reader;
5 ■ private final String filename;
6 ■
7 ■ // Constructor throws checked exception if file missing
8 ■ Config(String filename) throws FileNotFoundException {
9 ■ this.filename = filename;
10 ■ try {
11 ■ this.reader = new BufferedReader(new FileReader(filename));
12 ■ System.out.println("Config loaded from: " + filename);
13 ■ } catch (FileNotFoundException e) {
14 ■ System.out.println("Config constructor failed: " + filename);
15 ■ throw e; // re-throw – object construction fails
16 ■ }
17 ■ }
18 ■ String readSetting() throws IOException { return reader.readLine(); }
19 ■ @Override public void close() throws IOException {
20 ■ reader.close();
21 ■ System.out.println("Config closed.");
22 ■ }
23 ■ }
24 ■
25 ■ public class M11_ExceptionInConstructor {
26 ■ public static void main(String[] args) {
27 ■ String[] files = {"valid_config.properties", "missing_config.properties"};
28 ■ for (String f : files) {
29 ■ try (Config cfg = new Config(f)) {
30 ■ System.out.println("Setting: " + cfg.readSetting());
31 ■ } catch (FileNotFoundException e) {
32 ■ System.out.println("SKIP: " + e.getMessage());
33 ■ } catch (IOException e) {
34 ■ System.out.println("IO: " + e.getMessage());
35 ■ }
36 ■ }
37 ■ }

```

```
38 ■ }
```

■ Practice 12: Override Method with Exception Rules

```
1 ■ class Animal {
2 ■ // parent declares checked IOException
3 ■ void makeSound() throws java.io.IOException {
4 ■ System.out.println("Animal sound");
5 ■ }
6 ■ }
7 ■
8 ■ class Dog extends Animal {
9 ■ // VALID: narrower exception (FileNotFoundException extends IOException)
10 ■ @Override
11 ■ void makeSound() throws java.io.FileNotFoundException {
12 ■ System.out.println("Woof!");
13 ■ }
14 ■ }
15 ■
16 ■ class Cat extends Animal {
17 ■ // VALID: no exception (fewer than parent)
18 ■ @Override
19 ■ void makeSound() {
20 ■ System.out.println("Meow!");
21 ■ }
22 ■ }
23 ■
24 ■ // class Parrot extends Animal {
25 ■ // @Override void makeSound() throws Exception { } // COMPILE ERROR: broader
26 ■ // }
27 ■
28 ■ public class M12_OverrideException {
29 ■ public static void main(String[] args) {
30 ■ Animal[] animals = {new Dog(), new Cat()};
31 ■ for (Animal a : animals) {
32 ■ try {
33 ■ a.makeSound();
34 ■ } catch (java.io.IOException e) {
35 ■ System.out.println("IO: " + e.getMessage());
36 ■ }
37 ■ }
38 ■ }
39 ■ }
```

■ Practice 13: Recursive Function with Exception

```
1 ■ public class M13_RecursiveException {
2 ■ static long factorial(int n) {
3 ■ if (n < 0) throw new IllegalArgumentException("Factorial undefined for negative: " + n);
4 ■ if (n > 20) throw new IllegalArgumentException("Overflow risk for n > 20: " + n);
```

```

5 ■ if (n == 0 || n == 1) return 1;
6 ■ return n * factorial(n - 1);
7 ■ }
8 ■ // Safe recursive file listing
9 ■ static void listFiles(java.io.File dir, int depth) {
10 ■ if (!dir.exists()) throw new IllegalArgumentException("Does not exist: " + dir);
11 ■ if (!dir.isDirectory()) throw new IllegalArgumentException("Not a directory: " + dir);
12 ■ String indent = " ".repeat(depth);
13 ■ try {
14 ■ java.io.File[] files = dir.listFiles();
15 ■ if (files == null) throw new SecurityException("Access denied: " + dir);
16 ■ for (java.io.File f : files) {
17 ■ System.out.println(indent + f.getName());
18 ■ if (f.isDirectory()) listFiles(f, depth + 1);
19 ■ }
20 ■ } catch (SecurityException e) {
21 ■ System.out.println(indent + "[ACCESS DENIED]");
22 ■ }
23 ■ }
24 ■ public static void main(String[] args) {
25 ■ int[] inputs = {5, 10, 0, -1, 25};
26 ■ for (int n : inputs) {
27 ■ try {
28 ■ System.out.printf("%3d! = %d\n", n, factorial(n));
29 ■ } catch (IllegalArgumentException e) {
30 ■ System.out.println("Error: " + e.getMessage());
31 ■ }
32 ■ }
33 ■ try { listFiles(new java.io.File("."), 0); }
34 ■ catch (IllegalArgumentException e) { System.out.println("FS error: " + e.getMessage()); }
35 ■ }
36 ■ }

```

■ Practice 14: Menu-Driven Exception Handling

```

1 ■ import java.util.Scanner;
2 ■
3 ■ public class M14_MenuDriven {
4 ■ static int[] data = {10, 20, 30, 40, 50};
5 ■ static Scanner sc = new Scanner(System.in);
6 ■
7 ■ static void getElement() {
8 ■ System.out.print("Enter index (0-" + (data.length-1) + "): ");
9 ■ int idx = Integer.parseInt(sc.nextLine().trim());
10 ■ System.out.println("data[" + idx + "] = " + data[idx]);
11 ■ }
12 ■ static void divide() {
13 ■ System.out.print("Dividend: "); int a = Integer.parseInt(sc.nextLine().trim());
14 ■ System.out.print("Divisor: "); int b = Integer.parseInt(sc.nextLine().trim());
15 ■ System.out.println("Result: " + (a / b));

```

```

16 ■ }
17 ■ static void parseNumber() {
18 ■ System.out.print("Enter a number: ");
19 ■ int val = Integer.parseInt(sc.nextLine().trim());
20 ■ System.out.println("Parsed: " + val);
21 ■ }
22 ■
23 ■ public static void main(String[] args) {
24 ■ // Simulate menu choices for demo (replace with real Scanner.nextLine in interactive use)
25 ■ int[][] menu = {{1,1},{1,6},{2,10,0},{3,0},{4,0}};
26 ■ System.out.println("=== Demo mode (non-interactive) ===");
27 ■ for (int[] m : menu) {
28 ■ System.out.println("
29 ■ --- Menu choice " + m[0] + " ---");
30 ■ try {
31 ■ switch (m[0]) {
32 ■ case 1: System.out.println(m[1] < data.length && m[1] >= 0
33 ■ ? "data["+m[1]+"="+data[m[1]]
34 ■ : ""); if (m[1] >= data.length) throw new ArrayIndexOutOfBoundsException(m[1]); break;
35 ■ case 2: if (m[2] == 0) throw new ArithmeticException("/ by zero");
36 ■ System.out.println(m[1]/m[2]); break;
37 ■ case 3: System.out.println("Parsed: " + m[1]); break;
38 ■ case 4: System.out.println("Exiting."); return;
39 ■ }
40 ■ } catch (ArrayIndexOutOfBoundsException e) {
41 ■ System.out.println("[Array] Index out of range: " + e.getMessage());
42 ■ } catch (ArithmeticException e) {
43 ■ System.out.println("[Math] " + e.getMessage());
44 ■ } catch (Exception e) {
45 ■ System.out.println("[Error] " + e.getMessage());
46 ■ }
47 ■ }
48 ■ }

```

■ Practice 15: Login System with Exceptions

```

1 ■ import java.util.*;
2 ■
3 ■ class AuthException extends Exception {
4 ■ AuthException(String msg) { super(msg); }
5 ■ }
6 ■ class UserNotFoundException extends AuthException {
7 ■ UserNotFoundException(String user) { super("User not registered: " + user); }
8 ■ }
9 ■ class InvalidPasswordException extends AuthException {
10 ■ private final int remaining;
11 ■ InvalidPasswordException(String user, int remaining) {
12 ■ super("Wrong password for '" + user + "'. Attempts left: " + remaining);
13 ■ this.remaining = remaining;

```

```

14 ■ }
15 ■ int getRemaining() { return remaining; }
16 ■ }
17 ■ class AccountLockedException extends AuthException {
18 ■ AccountLockedException(String user) { super("Account locked due to too many failures: " +
19 ■ user); }
20 ■ }
21 ■ class AuthService {
22 ■ private static final Map<String, String> USERS =
23 ■ Map.of("alice", "pass123", "bob", "qwerty");
24 ■ private final Map<String, Integer> failCounts = new HashMap<>();
25 ■ private final Set<String> locked = new HashSet<>();
26 ■ private static final int MAX_ATTEMPTS = 3;
27 ■ void login(String user, String pass) throws AuthException {
28 ■ if (!USERS.containsKey(user)) throw new UserNotFoundException(user);
29 ■ if (locked.contains(user)) throw new AccountLockedException(user);
30 ■ if (!USERS.get(user).equals(pass)) {
31 ■ int fails = failCounts.merge(user, 1, Integer::sum);
32 ■ if (fails >= MAX_ATTEMPTS) locked.add(user);
33 ■ throw new InvalidPasswordException(user, MAX_ATTEMPTS - fails);
34 ■ }
35 ■ failCounts.remove(user);
36 ■ System.out.println("Login successful: " + user);
37 ■ }
38 ■ }
39 ■
40 ■ public class M15_LoginSystem {
41 ■ public static void main(String[] args) {
42 ■ AuthService auth = new AuthService();
43 ■ String[][] attempts = {
44 ■ {"alice", "pass123"}, {"bob", "wrong1"}, {"bob", "wrong2"},
45 ■ {"bob", "wrong3"}, {"bob", "qwerty"}, {"unknown", "any"},
46 ■ };
47 ■ for (String[] a : attempts) {
48 ■ try {
49 ■ auth.login(a[0], a[1]);
50 ■ } catch (AccountLockedException e) {
51 ■ System.out.println("[LOCKED] " + e.getMessage());
52 ■ } catch (InvalidPasswordException e) {
53 ■ System.out.println("[BAD_PASS] " + e.getMessage() + " (left: " + e.getRemaining() + ")");
54 ■ } catch (AuthException e) {
55 ■ System.out.println("[AUTH_ERR] " + e.getMessage());
56 ■ }
57 ■ }
58 ■ }
59 ■ }

```

■ Practice 16: Exception Propagation Across Methods

```

1 ■ public class M16_PropagationAcross {
2 ■ // Layer 3: data access
3 ■ static String fetchFromDB(int id) throws Exception {
4 ■ if (id <= 0) throw new IllegalArgumentException("ID must be positive: " + id);
5 ■ if (id > 100) throw new Exception("Record not found: id=" + id);
6 ■ return "Record#" + id;
7 ■ }
8 ■ // Layer 2: service
9 ■ static String getRecord(int id) throws Exception {
10 ■ System.out.println("Service: requesting id=" + id);
11 ■ return fetchFromDB(id); // propagates – no catch
12 ■ }
13 ■ // Layer 1: controller
14 ■ static String handleRequest(int id) {
15 ■ try {
16 ■ return getRecord(id);
17 ■ } catch (IllegalArgumentException e) {
18 ■ return "400 Bad Request: " + e.getMessage();
19 ■ } catch (Exception e) {
20 ■ return "404 Not Found: " + e.getMessage();
21 ■ }
22 ■ }
23 ■ public static void main(String[] args) {
24 ■ int[] ids = {1, 50, -1, 200};
25 ■ for (int id : ids) {
26 ■ System.out.println("Response: " + handleRequest(id));
27 ■ System.out.println();
28 ■ }
29 ■ }
30 ■ }

```

■ Practice 17: Handle Multiple Inputs Safely

```

1 ■ import java.util.*;
2 ■ public class M17_MultipleInputs {
3 ■ record ParseResult(String input, Object value, String error) {}
4 ■
5 ■ static ParseResult parseInput(String raw) {
6 ■ if (raw == null) return new ParseResult(null, null, "null input");
7 ■ raw = raw.trim();
8 ■ if (raw.isEmpty()) return new ParseResult(raw, null, "empty string");
9 ■ try {
10 ■ return new ParseResult(raw, Integer.parseInt(raw), null);
11 ■ } catch (NumberFormatException ignored) {}
12 ■ try {
13 ■ return new ParseResult(raw, Double.parseDouble(raw), null);
14 ■ } catch (NumberFormatException ignored) {}
15 ■ if (raw.equalsIgnoreCase("true") || raw.equalsIgnoreCase("false"))

```

```

16 ■ return new ParseResult(raw, Boolean.parseBoolean(raw), null);
17 ■ return new ParseResult(raw, raw, null); // treat as String
18 ■ }
19 ■
20 ■ public static void main(String[] args) {
21 ■ List<String> inputs = Arrays.asList("42", "3.14", "true", "hello", "", null, "100");
22 ■ for (String input : inputs) {
23 ■ ParseResult r = parseInput(input);
24 ■ if (r.error() != null)
25 ■ System.out.printf("Input %-10s → ERROR: %s\n", r.input(), r.error());
26 ■ else
27 ■ System.out.printf("Input %-10s → %s (%s)\n",
28 ■ r.input(), r.value(), r.value().getClass().getSimpleName());
29 ■ }
30 ■ }
31 ■ }

```

■ Practice 18: Suppressed Exception Demo

```

1 ■ public class M18_SuppressedExceptions {
2 ■ static class BrokenResource implements AutoCloseable {
3 ■ private final String name;
4 ■ private final boolean failOnClose;
5 ■ BrokenResource(String name, boolean failOnClose) {
6 ■ this.name = name;
7 ■ System.out.println("Opened: " + name);
8 ■ }
9 ■ void use() { System.out.println("Using: " + name); }
10 ■ @Override
11 ■ public void close() throws Exception {
12 ■ System.out.println("Closing: " + name);
13 ■ if (failOnClose) throw new Exception("Close failed: " + name);
14 ■ }
15 ■ }
16 ■
17 ■ public static void main(String[] args) {
18 ■ // try-with-resources: body exception + close exception → suppressed
19 ■ try (BrokenResource r = new BrokenResource("DB", true)) {
20 ■ r.use();
21 ■ throw new Exception("Body exception (primary)");
22 ■ } catch (Exception e) {
23 ■ System.out.println("Primary: " + e.getMessage());
24 ■ Throwable[] suppressed = e.getSuppressed();
25 ■ System.out.println("Suppressed count: " + suppressed.length);
26 ■ for (Throwable s : suppressed) {
27 ■ System.out.println(" Suppressed: " + s.getMessage());
28 ■ }
29 ■ }
30 ■ // Manual addSuppressed demo
31 ■ Exception primary = new Exception("Main error");

```



```

32 ■ primary.addSuppressed(new Exception("Cleanup error 1"));
33 ■ primary.addSuppressed(new Exception("Cleanup error 2"));
34 ■ try { throw primary; }
35 ■ catch (Exception e) {
36 ■ System.out.println("
37 ■ Manual primary: " + e.getMessage());
38 ■ for (Throwable s : e.getSuppressed())
39 ■ System.out.println(" Suppressed: " + s.getMessage());
40 ■ }
41 ■ }
42 ■ }

```

■ Practice 19: Calculator with Full Exception Handling

```

1 ■ import java.util.*;
2 ■ public class M19_Calculator {
3 ■ enum Op { ADD, SUB, MUL, DIV, MOD, POW }
4 ■
5 ■ static double calculate(double a, double b, Op op) {
6 ■ return switch (op) {
7 ■ case ADD -> a + b;
8 ■ case SUB -> a - b;
9 ■ case MUL -> a * b;
10 ■ case DIV -> {
11 ■ if (b == 0) throw new ArithmeticException("Division by zero");
12 ■ yield a / b;
13 ■ }
14 ■ case MOD -> {
15 ■ if (b == 0) throw new ArithmeticException("Modulo by zero");
16 ■ yield a % b;
17 ■ }
18 ■ case POW -> Math.pow(a, b);
19 ■ };
20 ■ }
21 ■
22 ■ static Op parseOp(String s) {
23 ■ return switch (s.toUpperCase()) {
24 ■ case "+", "ADD" -> Op.ADD;
25 ■ case "-", "SUB" -> Op.SUB;
26 ■ case "*", "MUL" -> Op.MUL;
27 ■ case "/", "DIV" -> Op.DIV;
28 ■ case "%", "MOD" -> Op.MOD;
29 ■ case "^", "POW" -> Op.POW;
30 ■ default -> throw new IllegalArgumentException("Unknown operator: " + s);
31 ■ };
32 ■ }
33 ■
34 ■ static double parseNum(String s) {
35 ■ try {
36 ■ return Double.parseDouble(s);

```

```

37 ■ } catch (NumberFormatException e) {
38 ■     throw new IllegalArgumentException("Not a number: " + s, e);
39 ■ }
40 ■ }
41 ■
42 ■ public static void main(String[] args) {
43 ■     String[][] exprs = {
44 ■         {"10", "2", "+"}, {"10", "2", "-"}, {"10", "2", "*"}, {"10", "0", "/"},
45 ■         {"10", "3", "%"}, {"2", "8", "^"}, {"abc", "2", "+"}, {"10", "2", "??"}
46 ■     };
47 ■     System.out.printf("%-6s %-4s %-6s %s\n", "A", "Op", "B", "Result");
48 ■     System.out.println("-".repeat(40));
49 ■     for (String[] e : exprs) {
50 ■         try {
51 ■             double a = parseNum(e[0]), b = parseNum(e[1]);
52 ■             Op op = parseOp(e[2]);
53 ■             double res = calculate(a, b, op);
54 ■             System.out.printf("%-6s %-4s %-6s = %.4f\n", e[0], e[2], e[1], res);
55 ■         } catch (ArithmeticException ex) {
56 ■             System.out.printf("%-6s %-4s %-6s → MATH ERROR: %s\n", e[0], e[2], e[1], ex.getMessage());
57 ■         } catch (IllegalArgumentException ex) {
58 ■             System.out.printf("%-6s %-4s %-6s → INPUT ERROR: %s\n", e[0], e[2], e[1], ex.getMessage());
59 ■         }
60 ■     }
61 ■ }
62 ■ }

```

■ Practice 20: Input Validation System

```

1 ■ import java.util.*;
2 ■ import java.util.regex.*;
3 ■
4 ■ class ValidationError {
5 ■     final String field, message;
6 ■     ValidationError(String field, String message) { this.field = field; this.message =
message; }
7 ■     public String toString() { return field + ": " + message; }
8 ■ }
9 ■
10 ■ class ValidationException extends RuntimeException {
11 ■     private final List<ValidationError> errors;
12 ■     ValidationException(List<ValidationError> errors) {
13 ■         super("Validation failed with " + errors.size() + " error(s)");
14 ■         this.errors = errors;
15 ■     }
16 ■     List<ValidationError> getErrors() { return errors; }
17 ■ }
18 ■
19 ■ class UserForm {
20 ■     String name, email, phone;

```

```

21 ■ int age;
22 ■ UserForm(String name, String email, String phone, int age) {
23 ■ this.name=name; this.email=email; this.phone=phone; this.age=age;
24 ■ }
25 ■ }
26 ■
27 ■ public class M20_InputValidationSystem {
28 ■ static void validate(UserForm form) {
29 ■ List<ValidationError> errs = new ArrayList<>();
30 ■ if (form.name == null || form.name.isBlank())
31 ■ errs.add(new ValidationError("name","required"));
32 ■ else if (form.name.length() < 2 || form.name.length() > 50)
33 ■ errs.add(new ValidationError("name","must be 2-50 chars"));
34 ■
35 ■ if (form.email == null || !form.email.matches("[\\w.+\\-]+@[\\w.\\-]+\\. [a-z]{2,}"))
36 ■ errs.add(new ValidationError("email","invalid format"));
37 ■
38 ■ if (form.phone != null && !form.phone.matches("\\+?[\\d\\s\\-]{7,15}"))
39 ■ errs.add(new ValidationError("phone","invalid format"));
40 ■
41 ■ if (form.age < 0 || form.age > 120)
42 ■ errs.add(new ValidationError("age","must be 0-120"));
43 ■
44 ■ if (!errs.isEmpty()) throw new ValidationException(errs);
45 ■ }
46 ■
47 ■ public static void main(String[] args) {
48 ■ List<UserForm> forms = List.of(
49 ■ new UserForm("Alice","alice@example.com","+91-9876543210",25),
50 ■ new UserForm("", "bad-email", "123",200),
51 ■ new UserForm("B","user@co.in",null,30)
52 ■ );
53 ■ for (UserForm f : forms) {
54 ■ try {
55 ■ validate(f);
56 ■ System.out.println("VALID: " + f.name);
57 ■ } catch (ValidationException e) {
58 ■ System.out.println("INVALID (" + f.name + "): " + e.getMessage());
59 ■ e.getErrors().forEach(err -> System.out.println(" • " + err));
60 ■ }
61 ■ System.out.println();
62 ■ }
63 ■ }
64 ■ }

```

■ HARD PRACTICE — 10 Programs

■ Practice 1: Mini Banking System with Exception Handling

[illegible]

[illegible]

```

99 ■ public static void main(String[] args) {
100 ■   BankService bank = new BankService();
101 ■   bank.open("ACC001", "Alice", 10_000);
102 ■   bank.open("ACC002", "Bob", 5_000);
103 ■
104 ■   Object[][] ops = {
105 ■     {"deposit", "ACC001", 3000.0},
106 ■     {"withdraw", "ACC001", 2000.0},
107 ■     {"transfer", "ACC001", "ACC002", 4000.0},
108 ■     {"withdraw", "ACC001", 20000.0}, // insufficient
109 ■     {"transfer", "ACC001", "ACC002", 60000.0}, // limit exceeded
110 ■     {"freeze", "ACC001"},
111 ■     {"deposit", "ACC001", 500.0}, // frozen
112 ■     {"unknown", "ACC999"}, // not found
113 ■   };
114 ■
115 ■   System.out.println("
116 ■   === TRANSACTIONS ===");
117 ■   for (Object[] op : ops) {
118 ■     try {
119 ■       switch ((String)op[0]) {
120 ■         case "deposit" -> bank.deposit((String)op[1], (double)op[2]);
121 ■         case "withdraw" -> bank.withdraw((String)op[1], (double)op[2]);
122 ■         case "transfer" -> bank.transfer((String)op[1], (String)op[2], (double)op[3]);
123 ■         case "freeze" -> bank.freeze((String)op[1]);
124 ■         default -> bank.get((String)op[1]);
125 ■       }
126 ■     } catch (InsufficientFundsException e) {
127 ■       System.out.printf("[%s] %s (shortfall=%.2f)%n",
128 ■         e.getCode(), e.getMessage(), e.getShortfall());
129 ■     } catch (AccountFrozenException | AccountNotFoundException |
130 ■       TransferLimitException e) {
131 ■       System.out.printf("[%s] %s%n", e.getCode(), e.getMessage());
132 ■     } catch (BankingException e) {
133 ■       System.out.printf("[%s] %s%n", e.getCode(), e.getMessage());
134 ■     }
135 ■   }
136 ■
137 ■   System.out.println("
138 ■   === STATEMENTS ===");
139 ■   bank.get("ACC001").printStatement();
140 ■   bank.get("ACC002").printStatement();
141 ■ }
142 ■ }

```

■ Practice 2: Custom Exception Hierarchy — Banking System

```

1 ■ import java.util.*;
2 ■
3 ■ /* Full exception hierarchy:

```

```

4 ■ BankingException (root, unchecked)
5 ■ ■■■■ AccountException
6 ■ ■ ■■■■ AccountNotFoundException
7 ■ ■ ■■■■ AccountFrozenException
8 ■ ■ ■■■■ DuplicateAccountException
9 ■ ■■■■ TransactionException
10 ■ ■ ■■■■ InsufficientFundsException
11 ■ ■ ■■■■ TransactionLimitException
12 ■ ■ ■■■■ DuplicateTransactionException
13 ■ ■■■■ AuthorizationException
14 ■ ■■■■ UnauthorizedAccessException
15 ■ ■■■■ SessionExpiredException
16 ■ */
17 ■
18 ■ // Root
19 ■ class BEx extends RuntimeException {
20 ■   enum Code {
ACC_NOT_FOUND, ACC_FROZEN, DUP_ACCOUNT, INSUF_FUNDS, TX_LIMIT, DUP_TX, UNAUTHORIZED, SESSION_EXP }
21 ■   private final Code code;
22 ■   BEx(String msg, Code code) { super(msg); this.code = code; }
23 ■   BEx(String msg, Code code, Throwable c) { super(msg,c); this.code=code; }
24 ■   Code getCode() { return code; }
25 ■   public String toString() { return "[" + code + "]" + getMessage(); }
26 ■ }
27 ■
28 ■ // Account branch
29 ■ class AccountEx extends BEx { AccountEx(String m, Code c) { super(m,c); } }
30 ■ class AccountNotFoundEx extends AccountEx {
31 ■   AccountNotFoundEx(String id) { super("Account "+id+" does not exist",
Code.ACC_NOT_FOUND); }
32 ■ }
33 ■ class AccountFrozenEx extends AccountEx {
34 ■   private final String reason;
35 ■   AccountFrozenEx(String id, String reason) {
36 ■     super("Account "+id+" is frozen", Code.ACC_FROZEN); this.reason=reason;
37 ■   }
38 ■   String getReason() { return reason; }
39 ■ }
40 ■ class DuplicateAccountEx extends AccountEx {
41 ■   DuplicateAccountEx(String id) { super("Account "+id+" already exists", Code.DUP_ACCOUNT);
}
42 ■ }
43 ■
44 ■ // Transaction branch
45 ■ class TransactionEx extends BEx { TransactionEx(String m, Code c) { super(m,c); } }
46 ■ class InsufficientFundsEx extends TransactionEx {
47 ■   private final double available, requested;
48 ■   InsufficientFundsEx(double a, double r) {
49 ■     super(String.format("Need %.2f but have %.2f",r,a), Code.INSUF_FUNDS);
50 ■     available=a; requested=r;

```

```

51 ■ }
52 ■ double getShortfall() { return requested - available; }
53 ■ }
54 ■ class TxLimitEx extends TransactionEx {
55 ■ TxLimitEx(double limit, double amt) {
56 ■ super(String.format("Amount %.2f exceeds limit %.2f",amt,limit), Code.TX_LIMIT);
57 ■ }
58 ■ }
59 ■ class DuplicateTxEx extends TransactionEx {
60 ■ DuplicateTxEx(String txId) { super("Duplicate transaction: "+txId, Code.DUP_TX); }
61 ■ }
62 ■
63 ■ // Authorization branch
64 ■ class AuthEx extends BEx { AuthEx(String m, Code c) { super(m,c); } }
65 ■ class UnauthorizedEx extends AuthEx {
66 ■ UnauthorizedEx(String user, String action) {
67 ■ super("User '"+user+"' not authorized for '"+action+"'", Code.UNAUTHORIZED);
68 ■ }
69 ■ }
70 ■ class SessionExpiredEx extends AuthEx {
71 ■ SessionExpiredEx(String sessionId) { super("Session expired: "+sessionId,
Code.SESSION_EXP); }
72 ■ }
73 ■
74 ■ public class H02_ExceptionHierarchy {
75 ■ record Session(String id, String role) {}
76 ■
77 ■ static void authorize(Session s, String action) {
78 ■ if (s == null) throw new SessionExpiredEx("null-session");
79 ■ if (!s.role().equals("ADMIN") && action.equals("FREEZE"))
80 ■ throw new UnauthorizedEx(s.id(), action);
81 ■ }
82 ■
83 ■ public static void main(String[] args) {
84 ■ Map<String, Double> accounts = new HashMap<>(Map.of("A1", 1000.0, "A2", 500.0));
85 ■ Set<String> txIds = new HashSet<>(), frozen = new HashSet<>();
86 ■ Session admin = new Session("admin-001", "ADMIN");
87 ■ Session user = new Session("user-001", "USER");
88 ■ double TX_LIMIT = 2000;
89 ■
90 ■ // Simulate operations
91 ■ Object[][] ops = {
92 ■ {admin, "DEPOSIT", "A1", "TX001", 800.0},
93 ■ {user, "WITHDRAW", "A1", "TX002", 1500.0},
94 ■ {user, "WITHDRAW", "A1", "TX002", 200.0}, // duplicate tx
95 ■ {user, "FREEZE", "A1", null, 0.0}, // unauthorized
96 ■ {admin, "FREEZE", "A1", null, 0.0},
97 ■ {user, "DEPOSIT", "A1", "TX003", 200.0}, // frozen
98 ■ {user, "WITHDRAW", "A1", "TX004", 3000.0}, // limit
99 ■ {user, "WITHDRAW", "A3", "TX005", 100.0}, // not found

```



```

100 ■ };
101 ■
102 ■ for (Object[] op : ops) {
103 ■     Session sess=(Session)op[0]; String action=(String)op[1];
104 ■     String acc=(String)op[2]; String txId=(String)op[3]; double amt=(double)op[4];
105 ■     System.out.printf("%-8s %-8s acc=%-3s tx=%-6s amt=%.2f → ",
106 ■         sess.id(), action, acc, txId, amt);
107 ■     try {
108 ■         authorize(sess, action);
109 ■         if (action.equals("FREEZE")) { frozen.add(acc); System.out.println("FROZEN"); continue;
110 ■     }
111 ■     if (!accounts.containsKey(acc)) throw new AccountNotFoundEx(acc);
112 ■     if (frozen.contains(acc)) throw new AccountFrozenEx(acc, "admin order");
113 ■     if (txId != null && txIds.contains(txId)) throw new DuplicateTxEx(txId);
114 ■     if (amt > TX_LIMIT) throw new TxLimitEx(TX_LIMIT, amt);
115 ■     double bal = accounts.get(acc);
116 ■     if (action.equals("WITHDRAW") && amt > bal) throw new InsufficientFundsEx(bal, amt);
117 ■     accounts.merge(acc, action.equals("DEPOSIT") ? amt : -amt, Double::sum);
118 ■     if (txId != null) txIds.add(txId);
119 ■     System.out.printf("OK BAL=%.2f\n", accounts.get(acc));
120 ■ } catch (InsufficientFundsEx e) {
121 ■     System.out.printf("%-20s shortfall=%.2f\n", e, e.getShortfall());
122 ■ } catch (BEx e) {
123 ■     System.out.println(e);
124 ■ }
125 ■ }
126 ■ }

```

■ Practice 3: File Parser with Exception Handling

```

1 ■ import java.io.*;
2 ■ import java.util.*;
3 ■
4 ■ class ParseException extends Exception {
5 ■     private final int lineNumber;
6 ■     private final String rawLine;
7 ■     ParseException(int lineNo, String raw, String reason) {
8 ■         super(String.format("Line %d: %s [raw: '%s']", lineNo, reason, raw));
9 ■         this.lineNumber = lineNo; this.rawLine = raw;
10 ■     }
11 ■     int getLineNumber() { return lineNumber; }
12 ■     String getRawLine() { return rawLine; }
13 ■ }
14 ■
15 ■ record Student(int id, String name, double gpa) {}
16 ■
17 ■ public class H03_FileParser {
18 ■     static final String CSV_CONTENT =
19 ■         "id,name,gpa

```

```
20 ■ " +
21 ■ "1,Alice,3.9
22 ■ " +
23 ■ "2,Bob,
24 ■ " + // missing gpa
25 ■ "3,,3.5
26 ■ " + // missing name
27 ■ "4,Charlie,5.0
28 ■ " + // gpa > 4.0
29 ■ "5,Dave,3.7
30 ■ " +
31 ■ "abc,Eve,3.2
32 ■ " + // non-numeric id
33 ■ "6,Frank,2.8
34 ■ ";
35 ■
36 ■ static Student parseLine(String line, int lineNo) throws ParseException {
37 ■     String[] parts = line.split(",", -1);
38 ■     if (parts.length != 3)
39 ■         throw new ParseException(lineNo, line, "expected 3 columns, got " + parts.length);
40 ■     int id;
41 ■     try { id = Integer.parseInt(parts[0].trim()); }
42 ■     catch (NumberFormatException e) {
43 ■         throw new ParseException(lineNo, line, "invalid id '" + parts[0].trim() + "'");
44 ■     }
45 ■     String name = parts[1].trim();
46 ■     if (name.isEmpty())
47 ■         throw new ParseException(lineNo, line, "name is required");
48 ■     double gpa;
49 ■     try {
50 ■         String gpaStr = parts[2].trim();
51 ■         if (gpaStr.isEmpty()) throw new ParseException(lineNo, line, "gpa is required");
52 ■         gpa = Double.parseDouble(gpaStr);
53 ■     } catch (NumberFormatException e) {
54 ■         throw new ParseException(lineNo, line, "invalid gpa '" + parts[2].trim() + "'");
55 ■     }
56 ■     if (gpa < 0 || gpa > 4.0)
57 ■         throw new ParseException(lineNo, line, "gpa must be 0.0-4.0, got " + gpa);
58 ■     return new Student(id, name, gpa);
59 ■ }
60 ■
61 ■ static List<Student> parseCSV(BufferedReader reader) {
62 ■     List<Student> students = new ArrayList<>();
63 ■     List<ParseException> errors = new ArrayList<>();
64 ■     String line;
65 ■     int lineNo = 0;
66 ■     try {
67 ■         String header = reader.readLine(); // skip header
68 ■         lineNo = 1;
69 ■         System.out.println("Header: " + header);
```

```

70 ■ while ((line = reader.readLine()) != null) {
71 ■     lineNo++;
72 ■     try {
73 ■         students.add(parseLine(line, lineNo));
74 ■     } catch (ParseException e) {
75 ■         errors.add(e); // collect errors, continue
76 ■     }
77 ■ }
78 ■ } catch (IOException e) {
79 ■     System.out.println("IO error at line " + lineNo + ": " + e.getMessage());
80 ■ }
81 ■ System.out.printf("Parsed %d records, %d errors%n", students.size(), errors.size());
82 ■ errors.forEach(e -> System.out.println(" SKIP: " + e.getMessage()));
83 ■ return students;
84 ■ }
85 ■
86 ■ public static void main(String[] args) {
87 ■     try (BufferedReader br = new BufferedReader(new StringReader(CSV_CONTENT))) {
88 ■         List<Student> students = parseCSV(br);
89 ■         System.out.println("
90 ■         === Valid Students ===");
91 ■         students.forEach(s -> System.out.printf(" %d %-10s GPA=%.1f%n",
92 ■             s.id(), s.name(), s.gpa()));
93 ■         double avgGpa = students.stream().mapToDouble(Student::gpa).average().orElse(0);
94 ■         System.out.printf("Average GPA: %.2f%n", avgGpa);
95 ■     } catch (IOException e) {
96 ■         System.out.println("Fatal IO: " + e.getMessage());
97 ■     }
98 ■ }
99 ■ }

```

■ Practice 4: Retry Mechanism Using Exception

```

1 ■ import java.util.*;
2 ■ import java.util.concurrent.*;
3 ■ import java.util.function.*;
4 ■
5 ■ class RetryException extends RuntimeException {
6 ■     private final int attempts;
7 ■     RetryException(String msg, int attempts, Throwable last) {
8 ■         super(msg + " after " + attempts + " attempts", last);
9 ■         this.attempts = attempts;
10 ■     }
11 ■     int getAttempts() { return attempts; }
12 ■ }
13 ■
14 ■ class RetryPolicy {
15 ■     final int maxAttempts;
16 ■     final long initialDelayMs;
17 ■     final double backoffMultiplier;

```

```

18 ■ final Set<Class<? extends Throwable>> retryOn;
19 ■
20 ■ @SafeVarargs
21 ■ RetryPolicy(int max, long delay, double mult, Class<? extends Throwable>... retryOn) {
22 ■     this.maxAttempts = max; this.initialDelayMs = delay;
23 ■     this.backoffMultiplier = mult;
24 ■     this.retryOn = new HashSet<>(Arrays.asList(retryOn));
25 ■ }
26 ■ boolean shouldRetry(Throwable t) {
27 ■     return retryOn.stream().anyMatch(c -> c.getInstance(t));
28 ■ }
29 ■ }
30 ■
31 ■ class Retry {
32 ■     static <T> T execute(Callable<T> task, RetryPolicy policy) {
33 ■         Throwable lastEx = null;
34 ■         long delay = policy.initialDelayMs;
35 ■         for (int attempt = 1; attempt <= policy.maxAttempts; attempt++) {
36 ■             try {
37 ■                 System.out.printf(" Attempt %d/%d...\n", attempt, policy.maxAttempts);
38 ■                 T result = task.call();
39 ■                 System.out.printf(" Success on attempt %d\n", attempt);
40 ■                 return result;
41 ■             } catch (Throwable e) {
42 ■                 lastEx = e;
43 ■                 System.out.printf(" Failed: %s\n", e.getMessage());
44 ■                 if (!policy.shouldRetry(e)) {
45 ■                     System.out.println(" Non-retryable exception – aborting");
46 ■                     break;
47 ■                 }
48 ■                 if (attempt < policy.maxAttempts) {
49 ■                     System.out.printf(" Waiting %dms before retry...\n", delay);
50 ■                     try { Thread.sleep(delay); } catch (InterruptedException ie) {
51 ■                         Thread.currentThread().interrupt();
52 ■                         throw new RetryException("Interrupted", attempt, ie);
53 ■                     }
54 ■                     delay = (long)(delay * policy.backoffMultiplier);
55 ■                 }
56 ■             }
57 ■         }
58 ■         throw new RetryException("All attempts exhausted", policy.maxAttempts, lastEx);
59 ■     }
60 ■ }
61 ■
62 ■ public class H04_RetryMechanism {
63 ■     static final Random RAND = new Random(42);
64 ■     static int callCount = 0;
65 ■
66 ■     // Simulates an unreliable remote service
67 ■     static String callRemoteAPI(String endpoint) throws Exception {

```

```

68 ■ callCount++;
69 ■ int roll = RAND.nextInt(5);
70 ■ if (roll < 2) throw new java.net.ConnectException("Connection refused: " + endpoint);
71 ■ if (roll < 3) throw new java.util.concurrent.TimeoutException("Timeout: " + endpoint);
72 ■ if (roll == 3) throw new IllegalArgumentException("Bad request (non-retryable)");
73 ■ return "Response from " + endpoint + " (call #" + callCount + ")";
74 ■ }
75 ■
76 ■ public static void main(String[] args) {
77 ■     RetryPolicy policy = new RetryPolicy(5, 100, 2.0,
78 ■     java.net.ConnectException.class,
79 ■     java.util.concurrent.TimeoutException.class);
80 ■
81 ■     String[] endpoints = {"/api/users", "/api/orders"};
82 ■     for (String ep : endpoints) {
83 ■         System.out.println("
84 ■         Calling: " + ep);
85 ■         try {
86 ■             String result = Retry.execute(() -> callRemoteAPI(ep), policy);
87 ■             System.out.println("Result: " + result);
88 ■         } catch (RetryException e) {
89 ■             System.out.println("GAVE UP: " + e.getMessage());
90 ■             System.out.println("Root cause: " + e.getCause().getMessage());
91 ■         }
92 ■     }
93 ■ }
94 ■ }

```

■ Practice 5: Exception Logging System

```

1 ■ import java.util.*;
2 ■ import java.util.concurrent.*;
3 ■ import java.time.*;
4 ■ import java.time.format.*;
5 ■
6 ■ // Severity levels
7 ■ enum Severity { DEBUG, INFO, WARN, ERROR, FATAL }
8 ■
9 ■ // Structured log record
10 ■ record LogRecord(Instant timestamp, Severity severity, String logger,
11 ■ String message, Throwable exception, Map<String,String> context) {}
12 ■
13 ■ // Log appender interface
14 ■ interface LogAppender {
15 ■     void append(LogRecord record);
16 ■ }
17 ■
18 ■ // Console appender with color
19 ■ class ConsoleAppender implements LogAppender {
20 ■     private static final Map<Severity, String> COLORS = Map.of(

```

```

21 ■ Severity.DEBUG, "■[37m", Severity.INFO, "■[32m",
22 ■ Severity.WARN, "■[33m", Severity.ERROR, "■[31m",
23 ■ Severity.FATAL, "■[35m"
24 ■ );
25 ■ private static final String RESET = "■[0m";
26 ■ private static final DateTimeFormatter FMT =
27 ■ DateTimeFormatter.ofPattern("HH:mm:ss.SSS").withZone(ZoneId.systemDefault());
28 ■ private final Severity minLevel;
29 ■ ConsoleAppender(Severity min) { this.minLevel = min; }
30 ■
31 ■ @Override public void append(LogRecord r) {
32 ■ if (r.severity().ordinal() < minLevel.ordinal()) return;
33 ■ String color = COLORS.getOrDefault(r.severity(), "");
34 ■ System.out.printf("%s[%s] [%-5s] [%s] %s%n",
35 ■ color, FMT.format(r.timestamp()), r.severity(),
36 ■ r.logger(), r.message(), RESET);
37 ■ if (!r.context().isEmpty())
38 ■ r.context().forEach((k,v) -> System.out.printf(" %s=%s%n",k,v));
39 ■ if (r.exception() != null) {
40 ■ System.out.printf(" %sException: %s: %s%n",
41 ■ color, r.exception().getClass().getSimpleName(),
42 ■ r.exception().getMessage(), RESET);
43 ■ StackTraceElement[] frames = r.exception().getStackTrace();
44 ■ for (int i = 0; i < Math.min(3, frames.length); i++)
45 ■ System.out.println(" at " + frames[i]);
46 ■ }
47 ■ }
48 ■ }
49 ■
50 ■ // In-memory appender (for testing / metrics)
51 ■ class InMemoryAppender implements LogAppender {
52 ■ final List<LogRecord> records = new CopyOnWriteArrayList<>();
53 ■ final Map<Severity, Long> counts = new ConcurrentHashMap<>();
54 ■ @Override public void append(LogRecord r) {
55 ■ records.add(r);
56 ■ counts.merge(r.severity(), 1L, Long::sum);
57 ■ }
58 ■ void printSummary() {
59 ■ System.out.println("
60 ■ === Log Summary ===");
61 ■ for (Severity s : Severity.values())
62 ■ System.out.printf(" %-5s: %d%n", s, counts.getOrDefault(s, 0L));
63 ■ System.out.println(" Total: " + records.size());
64 ■ }
65 ■ }
66 ■
67 ■ // Logger class
68 ■ class AppLogger {
69 ■ private final String name;
70 ■ private final List<LogAppender> appenders;

```

```

71 ■ AppLogger(String name, LogAppender... appenders) {
72 ■ this.name = name; this.appenders = Arrays.asList(appenders);
73 ■ }
74 ■ private void log(Severity sev, String msg, Throwable ex, Map<String,String> ctx) {
75 ■ LogRecord r = new LogRecord(Instant.now(), sev, name, msg, ex, ctx != null ? ctx :
Map.of());
76 ■ appenders.forEach(a -> a.append(r));
77 ■ }
78 ■ void debug(String msg) { log(Severity.DEBUG, msg, null, null); }
79 ■ void info(String msg) { log(Severity.INFO, msg, null, null); }
80 ■ void warn(String msg, Throwable ex) { log(Severity.WARN, msg, ex, null); }
81 ■ void error(String msg, Throwable ex, Map<String,String> ctx) { log(Severity.ERROR, msg,
ex, ctx); }
82 ■ void fatal(String msg, Throwable ex) { log(Severity.FATAL, msg, ex, null); }
83 ■ }
84 ■
85 ■ public class H05_ExceptionLogging {
86 ■ public static void main(String[] args) {
87 ■ ConsoleAppender console = new ConsoleAppender(Severity.DEBUG);
88 ■ InMemoryAppender memory = new InMemoryAppender();
89 ■ AppLogger log = new AppLogger("BankService", console, memory);
90 ■
91 ■ log.info("Service starting");
92 ■ log.debug("Config loaded: port=8080");
93 ■
94 ■ // Simulate various exception scenarios
95 ■ try {
96 ■ log.info("Processing payment");
97 ■ throw new ArithmeticException("Division by zero in fee calculation");
98 ■ } catch (ArithmeticException e) {
99 ■ log.error("Payment calculation failed", e,
100 ■ Map.of("txId", "TX-001", "amount", "1500.00", "currency", "INR"));
101 ■ }
102 ■
103 ■ try {
104 ■ throw new java.io.FileNotFoundException("config.properties");
105 ■ } catch (Exception e) {
106 ■ log.warn("Optional config missing – using defaults", e);
107 ■ }
108 ■
109 ■ try {
110 ■ throw new OutOfMemoryError("Java heap space");
111 ■ } catch (OutOfMemoryError e) {
112 ■ log.fatal("JVM out of memory – initiating graceful shutdown", e);
113 ■ }
114 ■
115 ■ memory.printSummary();
116 ■ }
117 ■ }

```

■ Practice 6: Multithreaded Exception Handling

```

1 ■ import java.util.*;
2 ■ import java.util.concurrent.*;
3 ■ import java.util.concurrent.atomic.*;
4 ■
5 ■ class TaskException extends Exception {
6 ■     private final int taskId;
7 ■     TaskException(int id, String reason, Throwable cause) {
8 ■         super("Task " + id + " failed: " + reason, cause);
9 ■         this.taskId = id;
10 ■     }
11 ■     int getTaskId() { return taskId; }
12 ■ }
13 ■
14 ■ record TaskResult(int taskId, String data, Exception error) {
15 ■     boolean isSuccess() { return error == null; }
16 ■ }
17 ■
18 ■ class TaskProcessor {
19 ■     private final ExecutorService pool;
20 ■     private final AtomicInteger successCount = new AtomicInteger();
21 ■     private final AtomicInteger failureCount = new AtomicInteger();
22 ■
23 ■     TaskProcessor(int threads) { pool = Executors.newFixedThreadPool(threads); }
24 ■
25 ■     Future<TaskResult> submit(int id, Callable<String> work) {
26 ■         return pool.submit(() -> {
27 ■             try {
28 ■                 Thread.sleep(new Random().nextInt(100));
29 ■                 String result = work.call();
30 ■                 successCount.incrementAndGet();
31 ■                 System.out.printf("[Thread %-20s] Task %2d: SUCCESS%n",
32 ■                     Thread.currentThread().getName(), id);
33 ■                 return new TaskResult(id, result, null);
34 ■             } catch (Exception e) {
35 ■                 failureCount.incrementAndGet();
36 ■                 System.out.printf("[Thread %-20s] Task %2d: FAILED - %s%n",
37 ■                     Thread.currentThread().getName(), id, e.getMessage());
38 ■                 return new TaskResult(id, null, new TaskException(id, e.getMessage(), e));
39 ■             }
40 ■         });
41 ■     }
42 ■
43 ■     void shutdown() { pool.shutdown(); }
44 ■     void printStats() {
45 ■         System.out.printf("%nStats: %d succeeded, %d failed%n",
46 ■             successCount.get(), failureCount.get());
47 ■     }
48 ■ }

```



```

49 ■
50 ■ public class H06_MultithreadedException {
51 ■     static final Random RAND = new Random();
52 ■
53 ■     static String simulateWork(int id) throws Exception {
54 ■         int roll = RAND.nextInt(4);
55 ■         return switch (roll) {
56 ■             case 0 -> throw new java.io.IOException("Network error for task " + id);
57 ■             case 1 -> throw new IllegalArgumentException("Bad data for task " + id);
58 ■             default -> "Result of task " + id;
59 ■         };
60 ■     }
61 ■
62 ■     public static void main(String[] args) throws Exception {
63 ■         TaskProcessor processor = new TaskProcessor(3);
64 ■         int TASK_COUNT = 10;
65 ■         List<Future<TaskResult>> futures = new ArrayList<>();
66 ■
67 ■         System.out.println("=== Submitting " + TASK_COUNT + " tasks ===");
68 ■         for (int i = 1; i <= TASK_COUNT; i++) {
69 ■             final int id = i;
70 ■             futures.add(processor.submit(id, () -> simulateWork(id)));
71 ■         }
72 ■
73 ■         processor.shutdown();
74 ■         processor.pool.awaitTermination(5, TimeUnit.SECONDS);
75 ■
76 ■         System.out.println("
77 ■         === Results ===");
78 ■         List<TaskResult> failed = new ArrayList<>();
79 ■         for (Future<TaskResult> f : futures) {
80 ■             TaskResult r = f.get();
81 ■             if (r.isSuccess()) System.out.println(" Task " + r.taskId() + ": " + r.data());
82 ■             else { failed.add(r); System.out.println(" Task " + r.taskId() + ": ERROR - " +
83 ■                 r.error().getMessage()); }
84 ■         }
85 ■         processor.printStats();
86 ■         if (!failed.isEmpty()) {
87 ■             System.out.println("
88 ■             === Failed Task Details ===");
89 ■             failed.forEach(r -> System.out.println(" " + r.error().getMessage()));
90 ■         }
91 ■     }

```

■ Practice 7: REST-Style Error Handling Simulation

```

1 ■ import java.util.*;
2 ■ import java.time.*;
3 ■

```

```

4 ■ // HTTP-style error response
5 ■ record ErrorResponse(int status, String code, String message,
6 ■ Instant timestamp, Map<String,Object> details) {}
7 ■
8 ■ // Exception hierarchy mapped to HTTP status codes
9 ■ class HttpException extends RuntimeException {
10 ■ private final int status;
11 ■ HttpException(String msg, int status) { super(msg); this.status = status; }
12 ■ int getStatus() { return status; }
13 ■ ErrorResponse toResponse(String code) {
14 ■ return new ErrorResponse(status, code, getMessage(), Instant.now(), Map.of());
15 ■ }
16 ■ }
17 ■ class BadRequestException extends HttpException {
18 ■ private final Map<String,String> fieldErrors;
19 ■ BadRequestException(Map<String,String> errs) {
20 ■ super("Validation failed", 400); fieldErrors=errs;
21 ■ }
22 ■ Map<String,String> getFieldErrors() { return fieldErrors; }
23 ■ }
24 ■ class NotFoundException extends HttpException { NotFoundException(String m) {
25 ■ super(m,404); } }
26 ■ class ConflictException extends HttpException { ConflictException(String m) {
27 ■ super(m,409); } }
28 ■ class InternalErrorException extends HttpException {
29 ■ InternalErrorException(String m, Throwable cause) { super(m, 500); initCause(cause); }
30 ■ }
31 ■ // Controller with global exception handler
32 ■ class UserController {
33 ■ private final Map<Integer, String> users = new HashMap<>(Map.of(1,"Alice",2,"Bob"));
34 ■ private final Set<String> emails = new HashSet<>(Set.of("alice@ex.com","bob@ex.com"));
35 ■
36 ■ ErrorResponse handleException(HttpException e) {
37 ■ String code = "ERR_" + e.getStatus();
38 ■ Map<String,Object> details = new HashMap<>();
39 ■ details.put("timestamp", Instant.now().toString());
40 ■ if (e instanceof BadRequestException bre) details.put("fields", bre.getFieldErrors());
41 ■ System.out.printf("[HTTP %d] %s\n", e.getStatus(), e.getMessage());
42 ■ return new ErrorResponse(e.getStatus(), code, e.getMessage(), Instant.now(), details);
43 ■ }
44 ■ ErrorResponse getUser(int id) {
45 ■ try {
46 ■ String user = users.get(id);
47 ■ if (user == null) throw new NotFoundException("User not found: " + id);
48 ■ System.out.println("200 OK: " + user);
49 ■ return new ErrorResponse(200, "OK", user, Instant.now(), Map.of());
50 ■ } catch (HttpException e) { return handleException(e); }
51 ■ }

```

```

52 ■
53 ■ ErrorResponse createUser(int id, String name, String email) {
54 ■ try {
55 ■ Map<String,String> errs = new LinkedHashMap<>();
56 ■ if (name == null || name.isBlank()) errs.put("name","required");
57 ■ if (email == null || !email.contains("@")) errs.put("email","invalid");
58 ■ if (!errs.isEmpty()) throw new BadRequestException(errs);
59 ■ if (users.containsKey(id)) throw new ConflictException("User " + id + " exists");
60 ■ if (emails.contains(email)) throw new ConflictException("Email taken: " + email);
61 ■ users.put(id, name); emails.add(email);
62 ■ System.out.println("201 Created: " + name);
63 ■ return new ErrorResponse(201,"CREATED",name,Instant.now(),Map.of("id",id));
64 ■ } catch (HttpException e) { return handleException(e); }
65 ■ catch (Exception e) { return handleException(new InternalErrorException("Unexpected
error",e)); }
66 ■ }
67 ■ }
68 ■
69 ■ public class H07_RestErrorHandling {
70 ■ public static void main(String[] args) {
71 ■ UserController ctrl = new UserController();
72 ■ System.out.println("=== GET /users/{id} ===");
73 ■ for (int id : new int[]{1, 99, 2}) ctrl.getUser(id);
74 ■ System.out.println("
75 ■ === POST /users ===");
76 ■ ctrl.createUser(3, "Charlie", "charlie@ex.com");
77 ■ ctrl.createUser(1, "Duplicate", "new@ex.com"); // conflict
78 ■ ctrl.createUser(4, "", "bad"); // validation
79 ■ ctrl.createUser(5, "Eve", "alice@ex.com"); // email taken
80 ■ }
81 ■ }

```

■ Practice 8: Fail-Safe System Design

```

1 ■ import java.util.*;
2 ■ import java.util.concurrent.*;
3 ■ import java.util.function.*;
4 ■
5 ■ // Circuit Breaker states
6 ■ enum CircuitState { CLOSED, OPEN, HALF_OPEN }
7 ■
8 ■ class CircuitBreaker {
9 ■ private volatile CircuitState state = CircuitState.CLOSED;
10 ■ private final AtomicInteger failures = new AtomicInteger();
11 ■ private volatile long openedAt;
12 ■ private final int failureThreshold;
13 ■ private final long resetTimeoutMs;
14 ■
15 ■ CircuitBreaker(int threshold, long timeoutMs) {
16 ■ failureThreshold = threshold; resetTimeoutMs = timeoutMs;

```

```

17 ■ }
18 ■
19 ■ <T> T execute(Callable<T> operation, Supplier<T> fallback) throws Exception {
20 ■     if (state == CircuitState.OPEN) {
21 ■         if (System.currentTimeMillis() - openedAt > resetTimeoutMs) {
22 ■             state = CircuitState.HALF_OPEN;
23 ■             System.out.println("[CB] HALF_OPEN - probing...");
24 ■         } else {
25 ■             System.out.println("[CB] OPEN - using fallback");
26 ■             return fallback.get();
27 ■         }
28 ■     }
29 ■     try {
30 ■         T result = operation.call();
31 ■         onSuccess();
32 ■         return result;
33 ■     } catch (Exception e) {
34 ■         onFailure();
35 ■         System.out.println("[CB] Failure #" + failures.get() + ": " + e.getMessage());
36 ■         if (state == CircuitState.OPEN) return fallback.get();
37 ■         throw e;
38 ■     }
39 ■ }
40 ■
41 ■ private void onSuccess() {
42 ■     failures.set(0);
43 ■     if (state == CircuitState.HALF_OPEN) { state = CircuitState.CLOSED;
44 ■         System.out.println("[CB] CLOSED"); }
45 ■ }
46 ■ private void onFailure() {
47 ■     if (failures.incrementAndGet() >= failureThreshold) {
48 ■         state = CircuitState.OPEN; openedAt = System.currentTimeMillis();
49 ■         System.out.println("[CB] OPEN - threshold reached");
50 ■     }
51 ■ }
52 ■ CircuitState getState() { return state; }
53 ■
54 ■ // Fail-safe service with fallback
55 ■ class InventoryService {
56 ■     private final CircuitBreaker cb = new CircuitBreaker(3, 2000);
57 ■     private final Map<String, Integer> remoteStock = Map.of("P1",100,"P2",50);
58 ■     private final Map<String, Integer> localCache = new HashMap<>(Map.of("P1",90,"P2",45));
59 ■     private int callCount = 0;
60 ■
61 ■     int getStock(String productId) {
62 ■         try {
63 ■             return cb.execute(
64 ■                 () -> {
65 ■                     callCount++;

```

```

66 ■ if (callCount % 3 != 0) // simulate intermittent failure
67 ■ throw new java.net.SocketTimeoutException("Remote timeout");
68 ■ System.out.println("Remote fetch for: " + productId);
69 ■ Integer stock = remoteStock.get(productId);
70 ■ if (stock == null) throw new NoSuchElementException("Unknown product: " + productId);
71 ■ localCache.put(productId, stock);
72 ■ return stock;
73 ■ },
74 ■ () -> {
75 ■ System.out.println("Fallback: cache for " + productId);
76 ■ return localCache.getDefault(productId, -1);
77 ■ }
78 ■ );
79 ■ } catch (NoSuchElementException e) { throw e; }
80 ■ catch (Exception e) { return localCache.getDefault(productId, -1); }
81 ■ }
82 ■ }
83 ■
84 ■ public class H08_FailSafeSystem {
85 ■ public static void main(String[] args) throws InterruptedException {
86 ■ InventoryService svc = new InventoryService();
87 ■ String[] products = {"P1", "P2", "P1", "P2", "P1", "P2", "P3", "P1"};
88 ■ for (String p : products) {
89 ■ try {
90 ■ System.out.printf("Stock[%s] = %d [CB: %s]%n",
91 ■ p, svc.getStock(p), "");
92 ■ } catch (NoSuchElementException e) {
93 ■ System.out.println("Product error: " + e.getMessage());
94 ■ }
95 ■ }
96 ■ }
97 ■ }

```

■ Practice 9: Large Input Validation System

[illegible]

[illegible]

```

64 ■ .check("email", r.email(), List.of(new NotNull<>(),
65 ■ new Pattern("[\\w.+\\-]+@[\\w\\.\\-]+\\. [a-z]{2,}", "is not a valid email")))
66 ■ .check("password", r.password(), List.of(new NotNull<>(), new Size(8, 64),
67 ■ new Pattern("[A-Z].*", "must contain uppercase"),
68 ■ new Pattern("[0-9].*", "must contain digit")))
69 ■ .check("age", (Number) r.age(), List.of(new NotNull<>(), new Range(18, 100)))
70 ■ .check("address.city", r.address().city(), List.of(new NotNull<>(), new Size(2, 50)))
71 ■ .check("address.zip", r.address().zip(), List.of(
72 ■ new Pattern("\\d{4,10}", "must be 4-10 digits")))
73 ■ .check("roles", r.roles() != null ? "present" : null,
74 ■ List.of(new NotNull<>()))
75 ■ .throwIfInvalid("UserRegistration");
76 ■ }
77 ■
78 ■ public static void main(String[] args) {
79 ■ List<UserRegistration> registrations = List.of(
80 ■ new UserRegistration("alice_92", "alice@example.com", "Secret99!", 25,
81 ■ new Address("MG Road", "Bangalore", "560001", "India"), List.of("USER")),
82 ■ new UserRegistration("ab", "not-an-email", "weak", 17,
83 ■ new Address("", "B", "abc", "India"), null),
84 ■ new UserRegistration("valid_user", "u@c.io", "Str0ng#Pass", 30,
85 ■ new Address("Park St", "Delhi", "110001", "India"), List.of("ADMIN", "USER"))
86 ■ );
87 ■
88 ■ for (UserRegistration reg : registrations) {
89 ■ try {
90 ■ validateRegistration(reg);
91 ■ System.out.println("VALID: " + reg.username());
92 ■ } catch (IllegalArgumentException e) {
93 ■ System.out.println("INVALID '" + reg.username() + "':");
94 ■ Arrays.stream(e.getMessage().split("; "))
95 ■ .forEach(err -> System.out.println(" • " + err));
96 ■ }
97 ■ System.out.println();
98 ■ }
99 ■ }
100 ■ }

```

■ Practice 10: Resilient System with Graceful Failure

[illegible]

```

11 ■ RateLimitException(String key) { super("Rate limit exceeded for: " + key); }
12 ■ }
13 ■
14 ■ // Simple token bucket rate limiter
15 ■ class RateLimiter {
16 ■     private final int maxTokens, refillPerSec;
17 ■     private double tokens;
18 ■     private Instant lastRefill = Instant.now();
19 ■     RateLimiter(int max, int refill) { maxTokens=max; refillPerSec=refill; tokens=max; }
20 ■     synchronized boolean tryAcquire() {
21 ■         refill();
22 ■         if (tokens >= 1) { tokens--; return true; }
23 ■         return false;
24 ■     }
25 ■     private void refill() {
26 ■         Instant now = Instant.now();
27 ■         double elapsed = Duration.between(lastRefill, now).toMillis() / 1000.0;
28 ■         tokens = Math.min(maxTokens, tokens + elapsed * refillPerSec);
29 ■         lastRefill = now;
30 ■     }
31 ■ }
32 ■
33 ■ // Bulkhead (concurrency limiter)
34 ■ class Bulkhead {
35 ■     private final Semaphore sem;
36 ■     Bulkhead(int max) { sem = new Semaphore(max); }
37 ■     boolean tryAcquire() { return sem.tryAcquire(); }
38 ■     void release() { sem.release(); }
39 ■ }
40 ■
41 ■ // Resilient service wrapper
42 ■ class ResilientService {
43 ■     private final String name;
44 ■     private final RateLimiter rateLimiter;
45 ■     private final Bulkhead bulkhead;
46 ■     private final long timeoutMs;
47 ■     private final ExecutorService exec = Executors.newCachedThreadPool();
48 ■
49 ■     ResilientService(String name, int rateLimit, int concurrency, long timeoutMs) {
50 ■         this.name = name;
51 ■         this.rateLimiter = new RateLimiter(rateLimit, rateLimit);
52 ■         this.bulkhead = new Bulkhead(concurrency);
53 ■         this.timeoutMs = timeoutMs;
54 ■     }
55 ■
56 ■     <T> T call(Callable<T> operation, Supplier<T> fallback) {
57 ■         if (!rateLimiter.tryAcquire()) {
58 ■             System.out.println "[" + name + "] Rate limited – fallback");
59 ■             return fallback.get();
60 ■         }

```



```

61 ■ if (!bulkhead.tryAcquire()) {
62 ■     System.out.println "[" + name + "] Bulkhead full – fallback");
63 ■     return fallback.get();
64 ■ }
65 ■ try {
66 ■     Future<T> future = exec.submit(operation);
67 ■     return future.get(timeoutMs, TimeUnit.MILLISECONDS);
68 ■ } catch (java.util.concurrent.TimeoutException e) {
69 ■     System.out.println "[" + name + "] Timeout – fallback");
70 ■     return fallback.get();
71 ■ } catch (ExecutionException e) {
72 ■     System.out.println "[" + name + "] Error: " + e.getCause().getMessage() + " – fallback");
73 ■     return fallback.get();
74 ■ } catch (InterruptedException e) {
75 ■     Thread.currentThread().interrupt();
76 ■     return fallback.get();
77 ■ } finally {
78 ■     bulkhead.release();
79 ■ }
80 ■ }
81 ■
82 ■ void shutdown() { exec.shutdownNow(); }
83 ■ }
84 ■
85 ■ public class H10_ResilientSystem {
86 ■     static final Random RAND = new Random();
87 ■
88 ■     static String fetchProduct(String id) throws Exception {
89 ■         Thread.sleep(RAND.nextInt(200)); // variable latency
90 ■         if (RAND.nextInt(3) == 0) throw new Exception("DB error for " + id);
91 ■         return "Product{" + id + ", price=" + (RAND.nextInt(90)+10) + "}";
92 ■     }
93 ■
94 ■     public static void main(String[] args) throws Exception {
95 ■         ResilientService svc = new ResilientService("ProductService",
96 ■             10, 3, 100); // 10 req/s, 3 concurrent, 100ms timeout
97 ■
98 ■         String[] products = {"P1", "P2", "P3", "P4", "P5", "P6", "P7", "P8"};
99 ■         List<Future<String>> futures = new ArrayList<>();
100 ■         ExecutorService clients = Executors.newFixedThreadPool(5);
101 ■
102 ■         System.out.println("=== Sending " + products.length + " requests ===");
103 ■         for (String p : products) {
104 ■             futures.add(clients.submit(() ->
105 ■                 svc.call(
106 ■                     () -> fetchProduct(p),
107 ■                     () -> "CACHED{" + p + "}"
108 ■                 )
109 ■             ));
110 ■         }

```

```
111 ■
112 ■ clients.shutdown();
113 ■ clients.awaitTermination(5, TimeUnit.SECONDS);
114 ■
115 ■ System.out.println("
116 ■ === Results ===");
117 ■ for (int i = 0; i < futures.size(); i++) {
118 ■ System.out.println(products[i] + " → " + futures.get(i).get());
119 ■ }
120 ■ svc.shutdown();
121 ■ }
122 ■ }
```

■ Glossary — Key Terms

- **Exception**
An object representing an abnormal runtime event that disrupts normal program execution.
- **Error**
A severe JVM-level problem (e.g., `OutOfMemoryError`) that applications should not attempt to catch.
- **Throwable**
The root superclass of all errors and exceptions; only `Throwable` instances can be thrown or caught.
- **Checked Exception**
An `Exception` subclass the compiler requires to be handled or declared with `throws` (e.g., `IOException`).
- **Unchecked Exception**
A `RuntimeException` subclass that the compiler does not require explicit handling for (e.g., `NullPointerException`).
- **try block**
A block of code enclosing statements that might throw an exception.
- **catch block**
A block that handles a specific exception type thrown by the preceding `try` block.
- **finally block**
A block that always executes after `try/catch` regardless of whether an exception occurred.
- **throw**
A statement that explicitly throws a `Throwable` instance from within a method.
- **throws**
A method-signature keyword declaring which checked exceptions the method may propagate.
- **Stack Trace**
A textual record of the call-stack frames at the moment an exception was thrown.
- **Exception Propagation**
The automatic bubbling of an unhandled exception up through the call stack to a handler.
- **Stack Unwinding**
Removal of call-stack frames as an exception propagates, executing `finally` blocks along the way.
- **Custom Exception**
A developer-defined class extending `Exception` or `RuntimeException` to model domain-specific failures.
- **try-with-resources**
A Java 7+ construct that automatically closes `AutoCloseable` resources when the `try` block exits.
- **AutoCloseable**
An interface with a `close()` method required for use in `try-with-resources`.
- **Multi-catch**
A Java 7+ syntax (`catch (A | B e)`) allowing one block to handle multiple exception types.
- **Chained Exception**
An exception wrapping another as its cause, preserving the full diagnostic context.
- **Suppressed Exception**
An exception silenced during auto-close in `try-with-resources`, attached to the primary exception.

- **fillInStackTrace()**

A Throwable method (called in the constructor) that captures the current call stack.

- **getMessage()**

Returns the detail message string set when the exception was constructed.

- **printStackTrace()**

Prints the exception's class, message, and full call-stack frames to standard error.

- **RuntimeException**

The parent class of all unchecked exceptions; represents programming errors, not external failures.

- **ArithmeticException**

Unchecked exception thrown for illegal arithmetic operations such as integer division by zero.

- **NullPointerException**

Unchecked exception thrown when code tries to use a null object reference.

- **ArrayIndexOutOfBoundsException**

Unchecked exception thrown when an array is accessed with an out-of-range index.

- **IOException**

Checked exception thrown when an I/O operation fails (file not found, read error, etc.).

- **NumberFormatException**

Unchecked exception thrown when parsing a string that does not have the appropriate numeric format.

- **ClassCastException**

Unchecked exception thrown when an object is cast to an incompatible type at runtime.

- **StackOverflowError**

Error thrown when the JVM call stack is exhausted, typically from unbounded recursion.